Chapter 20 — Cheat Sheets and Glossary

This last chapter squeezes Chapters 1–19 into pages you can scan in a minute or two: one **cheat sheet** per topic, **decision tables** for "which algorithm / which metric for this problem?", the **50 interview questions** that come up most often with short answers, and an alphabetical **glossary** of 911 terms. Nothing here is new. Every line points back to where the idea is taught properly, written like **§6.3** (= Chapter 6, section 6.3); if a line doesn't make sense, follow the reference. Every term and abbreviation used in the book also has a one-line plain-English definition in the glossary (§20.18) — look there first when a word on a sheet is new to you.

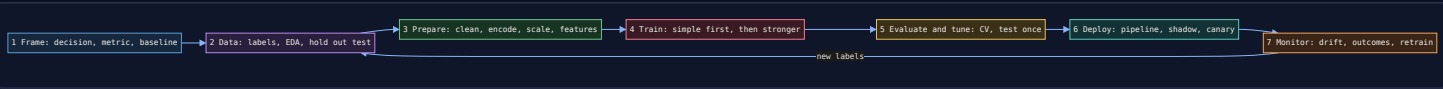
Every formula or number on a sheet that can be computed was recomputed by `code/cheatsheet_checks.py` (it also asserts the library defaults quoted here, for scikit-learn 1.9, PyTorch 2.x and NumPy 2.x):

```
OK  sheet_20_2_math
OK  sheet_20_3_api
OK  sheet_20_4_data_prep
OK  sheet_20_5_algorithms
OK  sheet_20_6_losses
OK  sheet_20_7_metrics
OK  sheet_20_8_validation
OK  sheet_20_9_deep_learning
OK  sheet_20_10_cnn_rnn_attention
OK  sheet_20_11_recommenders
OK  sheet_20_12_rl
OK  sheet_20_13_system_design
OK  sheet_20_14_production
OK  sheet_20_15_responsible
all 14 cheat-sheet checks passed
```

Chapter links: 1 · 2 · 3 · 4 · 5 · 6 · 7 · 8 · 9 · 10 · 11 · 12 · 13 · 14 · 15 · 16 · 17 · 18 · 19

Notation used throughout: x = feature vector, y = true label, ŷ = prediction, w = weights, b = bias, η = learning rate, σ = sigmoid, n = number of examples, d = number of features, k = number of neighbours / clusters / results (the context says which).

20.1 The ML workflow (§1.9, §4.10, §6.11, §15.3, §16.1)



Step	Do	Don't	Where
1. Frame	Name the decision the prediction drives, the task type (classification / regression / ranking / clustering), the metric tied to business value, and the **baseline** (current rule, human, "predict the mean")	Start by picking a model	§1.9, §15.3
2. Data	Find where labels come from and how late they arrive; set the test split aside **before** deep EDA (exploratory data analysis); check legality and privacy	Explore the test set; assume labels "exist"	§4.1, §4.2
3. Prepare	Dedupe, impossible values → NaN (the missing-value marker), impute (fill gaps), encode, scale — all inside one `Pipeline`	Fit anything on the full dataset (leakage)	§4.3–4.6, §4.10
4. Train	Linear/logistic model first, then a random forest and gradient boosting; deep nets for images/text/audio	Jump to the fanciest model	§5.1, §8.4
5. Evaluate	Cross-validate on train, tune on validation folds, report the test score **once**, next to the baseline, per segment	Tune on the test set; quote accuracy on imbalanced data	§6.1, §6.6, §6.9
6. Deploy	Ship preprocessing + model as one artifact (a single saved file); offline gate → shadow → canary → ramp, with a fallback	Re-implement features in another language (skew)	§16.2, §16.6
7. Monitor	Health → inputs → predictions → outcomes; investigate alerts; retrain through the same gates	Auto-promote on a drift alert	§16.8, §16.10

Diagnosing a model from two numbers (§1.10, §6.7):

Train error	Validation error	Diagnosis	First fixes
High	High (close)	Underfitting (high bias)	More capacity, better features, less regularisation, train longer
Low	Much higher	Overfitting (high variance)	More data, more regularisation, simpler model, early stopping, bagging
Low	Low (close)	Good fit	Test once, ship
Suspiciously perfect	Suspiciously perfect	Probably **leakage**	Check feature timestamps, splits, duplicates, top importances (§4.8)

Supervision types (§1.4): supervised (labels) · unsupervised (no labels: clusters, dimensions, anomalies) · semi-supervised (few labels + many unlabelled) · self-supervised (labels made from the data, e.g. next token) · reinforcement (rewards from actions). **Classification** predicts a category, **regression** a number (§1.5). **Batch** learning retrains on everything periodically; **online** learning updates with `partial_fit` (§1.7). **Instance-based** (k-NN) keeps the data; **model-based** keeps parameters (§1.8).

20.2 Math essentials (Ch 2)

Linear algebra (§2.1–2.3)

Idea	Formula	Worked value
Dot product	$a \cdot b = \Sigma\, a_i b_i = \|a\|\|b\|\cos\theta$	$[1,2,3]\cdot[4,5,6] =$ **32**
L2 / L1 / L∞ norm	$\sqrt{\Sigma a_i^2}$ · $\Sigma\|a_i\|$ · $\max\|a_i\|$	$\|[1,2,3]\|_2 = \sqrt{14} \approx 3.74$
Cosine similarity	$a \cdot b$ / ($\|a\|\|b\|$)	32 / (3.74·8.77) ≈ **0.975**
Linear model	$\hat{y} = w \cdot x + b$; batch: $\hat{y} = Xw + b$	shapes (n×d)(d,) → (n,)
Matrix product	(m×n)(n×p) → m×p; entry = row·column; **AB ≠ BA**	(100×20)(20×5) → 100×5
2×2 determinant / inverse	det = ad − bc; $A^{-1} = (1/\det)[[d, -b], [-c, a]]$	[[1,2],[3,4]]: det −2, inverse [[−2,1],[1.5,−0.5]]
Solve Ax = c	`np.linalg.solve(A, c)` (never `inv`)	$x_1+2x_2=5$, $3x_1+4x_2=11$ → x = [1, 2]
Eigenvector	$Av = \lambda v$	[[2,1],[1,2]]: λ = 3 on [1,1], λ = 1 on [1,−1]
SVD	$M = U\Sigma V^T$ (rotate, stretch, rotate)	top-k singular values = best rank-k approximation
Normal equation	$\theta = (X^TX)^{-1}X^Ty$ (use `lstsq`)	Ridge: $(X^TX + \alpha I)^{-1}X^Ty$

Calculus and optimisation (§2.4–2.5)

Idea	Formula
Power / chain rule	$(x^n)' = nx^{n-1}$ · $(f(g(x)))' = f'(g(x))\cdot g'(x)$
Sigmoid and derivative	$\sigma(z) = 1/(1 + e^{-z})$; $\sigma' = \sigma(1 - \sigma)$; $\sigma(0) = 0.5$, $\sigma'(0) = 0.25$ (its maximum)
Gradient	∇f = vector of partial derivatives; points **uphill**
Gradient descent	$w \leftarrow w - \eta\nabla L(w)$. On $f = (w-3)^2$, η = 0.1: 0 → 0.6 → 1.08 (each step keeps 80% of the gap)
MSE gradient	$\nabla = (2/n)X^T(Xw - y)$
Log-loss gradient	$\nabla = (1/n)X^T(\sigma(Xw) - y)$ — "features × (prediction − truth)"
Numerical gradient check	$f'(x) \approx (f(x+h) - f(x-h)) / 2h$
Convergence (MSE)	GD converges if $\eta < 2/\lambda_{max}$ of the Hessian (§5.4)

Probability and statistics (§2.6–2.11)

Idea	Formula / rule	Worked value
Bernoulli / binomial / normal	mean p, var p(1−p) · mean np, var np(1−p) · mean μ, var σ^2	P(3 of 10, p = 0.3) = C(10,3)·0.3^3·0.7^7 ≈ **0.2668**
68–95–99.7 rule	±1σ, ±2σ, ±3σ of a normal	P(z > 2) ≈ 2.3%
Expectation rules	E[aX+b] = aE[X]+b; $Var[aX+b] = a^2Var[X]$; independent: Var[X+Y] = Var X + Var Y	Var[3X] = 9·Var[X]
Conditional probability	$P(A\|B) = P(A \cap B)/P(B)$	
Bayes	$P(A\|B) = P(B\|A)P(A)/P(B)$ — posterior ∝ likelihood × prior	1% disease, 90% sensitivity, 5% FPR → P(sick|+) ≈ **0.154**; a 2nd positive → 0.766
z-score	$z = (x - \mu)/\sigma$	130 with μ 100, σ 15 → 2
Standard error (CLT, central limit theorem)	$\sigma/\sqrt{n}$; 4× the data halves it	σ 20, n 100 → 2
Population vs sample std	NumPy `ddof=0` (÷n), pandas `ddof=1` (÷(n−1))	
Hypothesis test	p-value = P(data this extreme | H_0); reject if p < α; Type I = α, power = 1 − β	p is **not** P(H_0 true)
Confidence interval	estimate ± critical value × SE (1.96 for 95%, large n)	bootstrap: resample, take 2.5th–97.5th percentiles
Multiple testing	Bonferroni: use α/m	20 tests at 0.05 → expect 1 false hit
Correlation	Pearson r (linear), Spearman (ranks, monotonic); r = 0 ≠ independent	$y = x^2$ can give r = 0

Information theory (§2.12)

Idea	Formula	Worked value (bits)
Entropy	$H(p) = -\Sigma\, p \log p$	fair coin 1; 90/10 coin 0.469; p = [0.7, 0.2, 0.1] → 1.157
Cross-entropy	$H(p, q) = -\Sigma\, p \log q \geq H(p)$	q = [0.5, 0.3, 0.2] → 1.280
KL divergence	$KL(p\|q) = H(p,q) - H(p) \geq 0$, **not symmetric**	0.123
Log loss (one-hot p)	$-(1/n)\ \Sigma\ [y \ln p + (1-y) \ln(1-p)]$	labels [1,0,1,1], p [0.9,0.2,0.6,0.99] → **0.2123** (nats)

20.3 NumPy, pandas and scikit-learn API (Ch 3, §4.10)

NumPy (§3.2)

Task	Code	Remember
Create	`np.array` , `zeros` , `ones` , `eye` , `arange` , `linspace(0, 1, 5)`	one dtype per array; `float32` = 4 bytes, `float64` = 8
Inspect	`a.shape` , `a.dtype` , `a.ndim` , `a.size` , `a.nbytes`	shape `(n,)` ≠ `(n, 1)`
Reshape	`a.reshape(3, 4)` , `a.reshape(-1, 1)` , `a[:, None]` , `a.T`	scikit-learn wants 2-D `X`
Index	`M[0]` , `M[:2, 1:3]` → **view**; `M[M > 5]` , `M[[0, 2]]` → **copy**	`.copy()` when you need independence
Reduce	`M.sum(axis=0)` (one per column), `axis=1` (one per row), `keepdims=True`	the named axis disappears
Broadcast	align shapes from the right; equal or 1 → OK	(3,1)+(1,4) → (3,4); (3,2)+(3,) → error
Matrix product	`A @ B` (not `*` , which is element-wise)	
Standardise columns	`(X - X.mean(axis=0)) / X.std(axis=0)`	fit statistics on training data only
Random	`rng = np.random.default_rng(42)` ; `rng.normal` , `rng.integers` , `rng.permutation`	seed once, draw everything from `rng`
Floats	`np.isclose(0.1 + 0.2, 0.3)`	`int8` 127 + 1 wraps to −128

pandas (§3.3)

Task	Code
Load / first look	`pd.read_csv(path)` ; `df.head()` , `df.info()` , `df.dtypes` , `df.describe()` , `df.isnull().sum()` , `df["c"].value_counts()`
Fix types	`pd.to_numeric(s, errors="coerce")` , `pd.to_datetime(s)` , `s.astype("category")`
Select	`df["c"]` (Series), `df[["a", "b"]]` (DataFrame), `df.loc[rows, cols]` (labels, **end inclusive**), `df.iloc[0:2]` (positions, end exclusive)
Filter	`df[(df.a > 1) & (df.b == "x")]` — `&` , `|` , `~` with parentheses
Missing values	`dropna()` , `fillna(v)` , `df.groupby(g)["x"].transform("median")` , `df["x_missing"] = df.x.isna()`
Group	`df.groupby("g").agg(n=("x", "size"), mean_x=("x", "mean"))` (one row per group); `transform` keeps the original length
Join / stack	`a.merge(b, on="k", how="left", validate="many_to_one")` ; `pd.concat([a, b])`
Vectorise	column arithmetic, `np.where` , `pd.cut` , `.str` , `.dt` — avoid row-wise `apply`
pandas 3	copy-on-write: assign (`df["c"] = df["c"].fillna(0)`), don't chain in-place calls

scikit-learn estimator API (§3.5, §4.10)

Convention	Meaning
`Model(hyper=...)`	hyperparameters go in the constructor
`fit(X, y)` → `self`	learns; learned attributes end in `_` (`coef_` , `mean_` , `classes_`)
`transform` / `fit_transform`	`fit_transform` on **train only**; `transform` on validation/test/production
`predict` , `predict_proba` , `decision_function` , `score`	`score` = accuracy (classifiers), R^2 (regressors)
`Pipeline` / `make_pipeline`	preprocessing + model = one estimator; refit per CV fold (no leakage), one artifact to save
`ColumnTransformer`	different preprocessing per column group; default `remainder="drop"`
`step__param`	address a parameter inside a pipeline, e.g. `prep__num__impute__strategy` , `model__C`
`set_output(transform="pandas")`	named DataFrame outputs for debugging
`joblib.dump(pipe, "m.joblib")`	save the whole pipeline; only load files you trust

```
pre = ColumnTransformer([
    ("num", make_pipeline(SimpleImputer(strategy="median", add_indicator=True), StandardScaler()), num_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols),
])
pipe = Pipeline([("pre", pre), ("model", LogisticRegression(max_iter=1000))])
search = GridSearchCV(pipe, {"model__C": [0.01, 0.1, 1, 10]}, cv=StratifiedKFold(5), scoring="average_precision")
search.fit(X_train, y_train)                # CV refits the preprocessing inside every fold
print(search.best_params_, search.score(X_test, y_test))   # test set used once
```

Defaults worth knowing (checked by the script): `LogisticRegression` C = 1.0 (L2), solver `lbfgs` · `RandomForestClassifier` `max_features="sqrt"` (regressor: 1.0) · `HistGradientBoosting*` learning_rate 0.1, `max_leaf_nodes` 31, `min_samples_leaf` 20 · `SVC` RBF kernel, C = 1, `gamma="scale"` · `KNeighbors*` k = 5, Minkowski p = 2 · `KMeans` `init="k-means++"` .

Module	Tools you'll use constantly
`model_selection`	`train_test_split` , `cross_val_score` , `cross_validate` , `KFold` , `StratifiedKFold` , `GroupKFold` , `TimeSeriesSplit` , `GridSearchCV` , `RandomizedSearchCV`
`preprocessing`	`StandardScaler` , `MinMaxScaler` , `RobustScaler` , `OneHotEncoder` , `OrdinalEncoder` , `TargetEncoder` , `PolynomialFeatures` , `KBinsDiscretizer` , `FunctionTransformer`
`impute`	`SimpleImputer` , `KNNImputer` , `IterativeImputer`
`metrics`	`accuracy_score` , `precision_score` , `recall_score` , `f1_score` , `roc_auc_score` , `average_precision_score` , `log_loss` , `brier_score_loss` , `mean_absolute_error` , `mean_squared_error` , `r2_score` , `ndcg_score` , `confusion_matrix` , `classification_report`
`calibration`	`CalibratedClassifierCV` , `calibration_curve`
`inspection`	`permutation_importance` , `PartialDependenceDisplay`

Frameworks (§3.7): scikit-learn (and XGBoost / LightGBM / CatBoost) for tables; PyTorch or TensorFlow/Keras for images, text, audio, GPUs and custom networks. Jupyter: seed first, Restart & Run All before sharing, move stable code into `.py` modules (§3.6).

20.4 Data preparation and feature engineering (Ch 4)

Labels (§4.1, §16.3)

Source	Speed	Weakness	Handle with
Natural outcomes (delivery time, chargeback)	Minutes → weeks	Late, incomplete, outcome ≠ truth	Train on mature rows; proxy labels for fresh data
Behaviour / implicit (clicks, watch time)	Seconds	Exposure and position bias	Log impressions and positions; correct bias (IPW, inverse propensity weighting)
Human annotation	Hours → days	Cost, reviewer quality	Versioned guidelines, Cohen's κ = (p_o − p_e)/(1 − p_e), gold questions, active learning
Weak supervision (labelling functions + label model)	Instant	Noisy, correlated rules	Snorkel-style label model; audit
LLM-assisted	Seconds	Correlated, systematic errors	Humans review samples, hard cases and per-subgroup slices

κ example: agreement 0.8, chance 0.5 → κ = **0.6**. Retrieval negatives: random (easy), in-batch (cheap, popularity-biased → logQ correction, which subtracts each item's log sampling probability), hard (sharp, some false negatives).

EDA checklist (§4.2): shape & types → missingness (and is it related to the target?) → distributions (skew, sentinels like 999 / 9,999,999, cardinality) → target balance → feature–target and feature–feature relationships (**Spearman** for skewed data) → duplicates, impossible values, leaky IDs. Do detailed EDA on the training split.

Cleaning (§4.3)

Problem	Default fix
Exact duplicates	`drop_duplicates()` , then check the entity key is unique; dedupe **across** splits
Impossible values (age 999)	Set to NaN — they are errors, not outliers
Outliers	IQR (interquartile range) fences (Q1 − 1.5·IQR, Q3 + 1.5·IQR), |z| > 3, or log first; cap at training-set percentiles; never blindly delete (fraud *is* the outlier)
Missing: MCAR / MAR / MNAR (missing completely at random / at random / not at random)	Median (numbers) or `"missing"` (categories) **+ missing indicator**; KNN or iterative imputation when other columns predict it; HistGradientBoosting/XGBoost/LightGBM handle NaN natively
Judging an imputer	Hide known values, measure error, and compare downstream CV scores

Encoding (§4.4)

Encoder	Use for	Risk
One-hot (`handle_unknown="ignore"`)	Nominal, low–moderate cardinality	Width; dummy trap for unregularised OLS (`drop="first"`)
Ordinal	Ordered categories; fine for trees	Fake order for linear models / k-NN
Target encoding (`TargetEncoder`)	High cardinality	**Leakage** unless cross-fitted and smoothed
Hashing (`FeatureHasher`)	Unbounded IDs, streaming	Collisions, not reversible
Embeddings	IDs in deep models / recommenders	Needs data per ID; versioning

Scaling (§4.5)

Scaler	Formula	Use when
Standard	(x − mean)/std	Default for linear models, SVMs, neural nets, PCA, k-NN, k-means
Min-max	(x − min)/(max − min)	Bounded inputs (pixels ÷ 255), no big outliers
Robust	(x − median)/IQR	Outliers you keep: [10,11,11,12,12,13,100] → 100 ↦ (100 − 12)/1.5 = 58.67
Log / power	log1p(x), `PowerTransformer`	Heavy right tails (income, counts) — scaling alone never fixes skew
None	—	Trees and tree ensembles

Feature creation (§4.6)

Technique	Example
Interactions / polynomials	price × quantity; `PolynomialFeatures(2)` on 10 features → 65 columns
Binning	age buckets let a linear model fit a U-shape
Log transform	`log1p(income)`
Dates and cycles	day of week, is_weekend, "days since signup"; hour h → (sin 2πh/24, cos 2πh/24): hour 6 → (1, 0)
Text	Bag-of-words, TF-IDF: idf(t) = ln((1+n)/(1+df)) + 1, rows L2-normalised (n = 4, df = 2 → 1.511)
Aggregates and ratios	orders in last 30 min, tickets per month of tenure — must be **point-in-time correct**
Production groups	**entity** (batch), **behavioural aggregates** (streaming), **context** (request time), **cross** (pairs) — §4.6, §16.4

Splits (§4.7): random (i.i.d. data) · **stratified** (classification, keeps class ratios) · **time-based** (anything predicting the future) · **group** (all rows of one user/patient together, to generalise to new entities). Millions of rows → 98/1/1 is fine; a few thousand → cross-validation.

Leakage checklist (§4.8, §16.3): (1) would I know this value at prediction time? (2) split first, then fit everything inside a `Pipeline`; (3) time/group splits when deployment needs them; (4) dedupe across splits; (5) aggregates whose window includes the label moment; (6) label-adjacent fields (refund flag, resolution code); (7) distrust any single dominant feature or a score far above published baselines.

Imbalance (§4.9)

Technique	Effect on probabilities
Better metrics (precision, recall, PR-AUC)	—
Threshold moving	None — often all you need
Class weights (`class_weight="balanced"`)	Inflated → recalibrate
Negative downsampling (keep fraction w)	Inflated → **p_true = p·w / (p·w + 1 − p)**; w = 0.1, p = 0.5 → 0.091
Oversampling / SMOTE (a + u·(b − a) between minority neighbours)	Inflated; resample the **training** data only, after splitting

---

20.5 Every algorithm in one table (Ch 5, 7–14)

n = rows, d = features, k = neighbours/clusters, B = trees, M = boosting rounds, T = sequence length, h = hidden width. Complexities are rough orders of magnitude for intuition, not exact bounds (big-O notation: O(nd) means cost grows in proportion to n × d).

Algorithm	What it does	Use when	Key hyperparameters	Pros	Cons	Cost (train / predict)	§
Linear regression (OLS)	$\hat{y} = w \cdot x + b$ minimising MSE	Roughly linear trends, baselines, explaining effects	none (fit_intercept)	Fast, interpretable, extrapolates a trend	Misses curves; outliers; multicollinearity	$O(nd^2 + d^3)$ solver or $O(nd)$/epoch GD / $O(d)$	5.2–5.3
Ridge / Lasso / ElasticNet	OLS + L2 / L1 / both penalty	Many or correlated features; want sparsity (L1)	α (and `l1_ratio`)	Less overfitting; Lasso selects features	Must scale; α tuned by CV	As OLS / $O(d)$	5.7
Polynomial regression	Linear model on powers/products	Smooth curves, few features	degree	Same solvers	Feature explosion; wild at the edges	As OLS on expanded d	5.6
Logistic regression	$p = \sigma(w \cdot x + b)$, log-loss	Binary baseline, calibrated probabilities, sparse text	C (= 1/strength), class_weight	Convex, fast, odds ratios, roughly calibrated	Linear boundary	$O(nd)$ per iteration / $O(d)$	5.8
Softmax regression	K linear scores → softmax	Mutually exclusive classes	C	Coherent probabilities	Linear boundaries	$O(ndK)$ / $O(dK)$	5.9
k-NN	Vote/average of k nearest points	Small low-dimensional data, similarity search	k, metric, weights	No training, any boundary	Slow prediction, needs scaling, curse of dimensionality	$O(1)$ / $O(nd)$ brute force	8.1
Linear SVM	Max-margin hyperplane, hinge loss	High-dimensional sparse text	C	Fast, strong on text	No probabilities	$\sim O(nd)$ / $O(d)$	8.2
Kernel SVM (RBF)	Max margin in an implicit feature space	Small/medium data with curved boundaries	C, γ (tune jointly, log scale)	Convex, flexible, compact (support vectors)	Slow beyond tens of thousands of rows; scaling; no native probabilities	$\sim O(n^2 d)$ to $O(n^3 d)$ / $O(n_{SV} \cdot d)$	8.2
SVR	Fit inside an ε-tube	Small-data regression with noise	C, ε, γ	Ignores small errors	As kernel SVM	As SVM	8.2
Naive Bayes	Bayes + "features independent given class"	Text, tiny data, streaming baselines	α (smoothing), variant (Multinomial/Bernoulli/Gaussian)	Fastest to train, few data needed	Over-confident probabilities, no interactions	$O(nd)$ / $O(dK)$	8.3
Decision tree (CART)	Greedy binary splits maximising impurity decrease	Readable rules, quick look	max_depth, min_samples_leaf, ccp_alpha	Interpretable, no scaling, non-linear	High variance, axis-aligned, no extrapolation	$\sim O(nd \log n)$ / $O(\text{depth})$	7.1
Random forest	Bagged deep trees + random feature subset per split	Strong default with little tuning	n_estimators, max_features, max_depth / min_samples_leaf	Robust, parallel, free OOB score, more trees never hurt	Bigger/slower; less accurate than tuned boosting; no extrapolation	B × tree / $O(B \cdot \text{depth})$	7.3
AdaBoost	Re-weight misclassified samples; weighted vote of stumps	Clean data, simple baseline	n_estimators, learning_rate	Simple	Sensitive to noise/outliers (exponential loss)	M × stump / $O(M)$	7.4
Gradient boosting (sklearn HGB, XGBoost, LightGBM, CatBoost)	Sequential shallow trees fitted to the negative gradient	**Best default for tabular data**	learning_rate, n_estimators (+ early stopping), depth/leaves, min leaf size, subsample, L1/L2	Top tabular accuracy, native NaN/categoricals	Overfits with too many rounds; sequential	M × histogram tree ≈ $O(M \cdot nd)$ / $O(M \cdot \text{depth})$	7.4, 7.7
Voting / stacking	Combine different model families (stacking: meta-model on out-of-fold predictions)	Diverse models with different errors	base models, meta-model, cv	Reduces risk	More latency and upkeep; not guaranteed to win	Sum of models	7.5
k-means	Assign to nearest centroid, move centroids, repeat	Round, similar-sized clusters	k, n_init (k-means++)	Fast, scales (MiniBatch)	Must pick k; spherical assumption; outliers	$O(nkd)$ per iteration	9.1

Algorithm	What it does	Use when	Key hyperparameters	Pros	Cons	Cost (train / predict)	§
Agglomerative	Merge closest clusters bottom-up (dendrogram)	Small data, nested structure	linkage, n_clusters or cut height	No k up front, interpretable tree	$O(n^2)$ memory	$\geq O(n^2)$	9.2
DBSCAN / HDBSCAN	Clusters = dense regions; noise = −1	Odd shapes, outliers, unknown k	eps, min_samples (HDBSCAN: min_cluster_size)	Any shape, labels noise	One global density (DBSCAN); eps hard in high-d	~O(n log n) with an index	9.3
Gaussian mixture (EM)	Soft assignment to k Gaussians	Elliptical/overlapping clusters, need probabilities	k (by BIC), covariance_type	Soft, elliptical	Local optima; k to choose	$O(nkd^2)$ per EM step (full cov.)	9.4
PCA	Project onto top eigenvectors of the covariance	Compression, denoising, pre-step, quick 2-D look	n_components (e.g. 0.95 variance)	Fast, deterministic, `transform`	Linear only; scale first	full SVD ~$O(nd \cdot \min(n,d))$	9.5
t-SNE / UMAP	Non-linear 2-D maps preserving neighbours	Visualisation only	perplexity / n_neighbors, min_dist	Reveals clusters	Sizes and gaps meaningless; seed-dependent	t-SNE ~O(n log n) (Barnes-Hut)	9.6
Isolation Forest	Anomalies isolate in few random splits	Multivariate anomaly default	n_estimators, max_samples (256), contamination	Fast, no scaling	Misses anomalies visible only in combinations	~linear	9.7
LOF / One-Class SVM	Local density vs neighbours / boundary around normal data	Mixed densities / novelty on clean data	k / ν, γ	Local / flexible	$O(n^2)$-ish / slow on big n	—	9.7
Apriori / FP-Growth	Frequent itemsets → rules ranked by lift	Market-basket analysis	min_support, min_confidence	Readable rules	Many trivial rules	Grows with candidates	9.8
MLP (neural network)	Layers of h = f(Wx + b), trained by backprop	Large data; building block of deep nets	depth, width, lr, batch size, dropout, weight decay	Learns features, very flexible	Data-hungry, many knobs; usually loses to GBDT on small tables	O(params) per example per step	10.1
CNN	Shared convolution kernels + pooling	Images, spatial grids	depth, channels, kernel size, augmentation	Few parameters, translation equivariance	Needs data or a pretrained backbone	$O(k^2 \cdot C_{in} \cdot C_{out} \cdot H \cdot W)$ per layer	11.1–11.2
RNN / LSTM / GRU	Hidden state carried step by step	Streaming/small sequence models, time series	hidden size, layers, clipping	Constant memory per step	Sequential (no parallelism over time)	$O(T \cdot h^2)$	12.2–12.4
Transformer	Self-attention + feed-forward blocks	Text, speech, vision at scale	layers, d_model, heads, context length	Parallel, O(1) path between tokens	$O(T^2 \cdot d)$ attention cost	$O(T^2 d + Td^2)$ per layer	12.7
Matrix factorisation (SGD/ALS)	$\hat{r} = \mu + b_u + b_i + p_u \cdot q_i$	Collaborative filtering baseline	k factors, λ, (α for implicit)	Strong, compact embeddings	Cold start; no context	SGD $O(\|R\| \cdot k)$/epoch	13.5
Two-tower	User and item encoders meeting at a dot product	Retrieval over millions of items	embedding dim, temperature, negatives	Items precomputed → ANN	Weak feature crosses; versioning	Per request: user tower + ANN	13.7
Multi-armed bandits (ε-greedy, UCB, Thompson)	Explore/exploit over k arms	Headlines, thumbnails, new-item traffic	ε, c, priors	Less regret than A/B	Non-stationarity handling	O(k) per step	14.4
Q-learning / SARSA	TD control on a Q-table	Small discrete MDPs	α, γ, ε schedule	Model-free, simple	Table must fit; sample-hungry	O(1) per update	14.7
DQN	Neural Q-function + replay + target network	Discrete actions, large state spaces	replay size, target sync C, ε	Scales to pixels	Unstable (deadly triad)	Network forward/backward	14.8
Policy gradient / actor-critic / PPO	Gradient ascent on expected return	Continuous actions, RLHF	lr, clip ε, entropy bonus, GAE λ	Stochastic and continuous policies	High variance, on-policy	Network forward/backward	14.9
ARIMA / ETS	AR + differencing + MA / smoothing	Single short time series	(p, d, q), seasonality	Built-in intervals, little data	One series at a time, few extra features	Small	12.10

20.6 Loss functions (Ch 2, 5, 8, 10, 13, 14)

Loss	Formula (one example)	Task / model	Notes	§
MSE (squared error)	$(\hat{y} - y)^2$	Regression, linear regression, neural nets	Punishes big errors; optimal constant = **mean**; MLE under Gaussian noise	5.2, 10.3
MAE (L1)	$\lvert\hat{y} - y\rvert$	Robust regression	Optimal constant = **median**	6.2, 10.3
Huber	$\frac{1}{2}e^2$ if $\lvert e\rvert \le \delta$, else $\delta(\lvert e\rvert - \frac{1}{2}\delta)$	Regression with outliers; DQN	Quadratic near 0, linear far away	10.3, 14.8
Quantile (pinball)	$\max(\tau \cdot e, (\tau - 1) \cdot e)$, $e = y - \hat{y}$	Prediction intervals, p90 forecasts	$\tau = 0.9$ → 90th percentile	12.10
Log loss / binary cross-entropy	$-[y \ln p + (1-y) \ln(1-p)]$	Logistic regression, sigmoid outputs, multi-label	Convex for LR; gradient at logit = $p - y$; PyTorch: `BCEWithLogitsLoss` (logits in)	5.8, 10.3
Categorical cross-entropy	$-\ln p_{true}$	Softmax classifiers, language models	Uniform guess = $\ln K$ ($\ln 10 = 2.303$); gradient = $p - \text{one_hot}$; PyTorch `CrossEntropyLoss` takes logits + class indices	5.9, 10.3
Hinge	$\max(0, 1 - y \cdot f(x))$, $y \in \{-1, +1\}$	SVM	Zero for points beyond the margin	8.2
ε-insensitive	$\max(0, \lvert y - f(x)\rvert - \varepsilon)$	SVR	Errors inside the tube are free	8.2
Exponential	$e^{-y \cdot f(x)}$	AdaBoost	Explains its outlier sensitivity	7.4
Regularised squared error	$\Sigma(r - \hat{r})^2 + \lambda(\lVert p\rVert^2 + \lVert q\rVert^2 + b^2)$ over observed cells	Matrix factorisation	Implicit version weights every cell by confidence	13.5
Sampled softmax / InfoNCE	$-\ln [e^{(s^+/\tau)} / \Sigma e^{(s/\tau)}]$ over in-batch items (logits − log Q for popularity)	Two-tower retrieval, CLIP-style contrastive learning	Temperature $\tau \approx 0.05$–0.1	13.7, 18.6
Skip-gram negative sampling	$-\ln \sigma(v \cdot u^+) - \Sigma \ln \sigma(-v \cdot u^-)$	word2vec, item2vec	Negatives $\propto$ frequency$^{0.75}$	12.9, 13.6
Pairwise / listwise ranking	pairwise: which of two items is better (RankNet/LambdaMART); listwise: optimise NDCG-like list metrics	Learning-to-rank	Pointwise = predict each label independently	17.3
Regularisation terms	L2: $\lambda\Sigma w^2$ · L1: $\lambda\Sigma\lvert w\rvert$	Any	L1 → exact zeros; L2 → shrinkage; AdamW = decoupled L2	5.7, 10.7
TD / Q-learning target	$(r + \gamma \max_{a'} Q(s',a'; \theta^-) - Q(s,a; \theta))^2$ (Huber in DQN)	Value-based RL	θ^- = frozen target network	14.8
Policy gradient	$-\log \pi(a\lvert s) \cdot A_t$	REINFORCE, actor-critic	$A = G - V(s)$ or the TD error	14.9
Reward model (preferences)	$-\ln \sigma(s_{preferred} - s_{rejected})$	RLHF	PPO then maximises reward − $\beta \cdot KL(\pi \,\|\, \pi_{SFT})$	14.10
Fairness penalty	loss + $\lambda \cdot$(mean score A − mean score B)^2	In-processing bias mitigation	Larger λ → more parity, less accuracy	19.4

Rule of thumb: pick the output layer and the loss together — identity + MSE/MAE/Huber (regression), sigmoid + BCE (binary, multi-label), softmax + cross-entropy (one label out of K) (§10.2). Never train on accuracy: its gradient is zero almost everywhere.

20.7 Metrics: formula and when to use (Ch 6, 9, 11, 13, 16)

Regression (§6.2, §16.7) — worked on $y = (30, 25, 40, 35, 50)$, $\hat{y} = (28, 27, 35, 35, 60)$:

Metric	Formula	Value	Use when
MAE	$(1/n)\Sigma\lvert e\rvert$	3.8	Cost grows linearly with error; label outliers shouldn't dominate
MSE / RMSE	$(1/n)\Sigma e^2$ / $\sqrt{\text{MSE}}$	26.6 / 5.16	Big misses are disproportionately bad (a 40-min miss adds 16× what a 10-min miss adds to MSE)
R^2	$1 - \Sigma e^2/\Sigma(y - \bar{y})^2$	0.641	Share of variance explained; 0 = predicting the mean, < 0 = worse
MAPE	$(100/n)\Sigma\lvert e/y\rvert$	—	Relative error; breaks near y = 0
p90 absolute error	90th percentile of $\lvert e\rvert$	—	Promises made on the tail (ETAs)
Per-cohort error	Any of the above per segment	—	Always: averages hide failing segments

Classification (§6.3, §6.5) — worked on 1,000 transactions, 100 fraud, 80 flagged, 60 correct (TP 60, FP 20, FN 40, TN 880):

Metric	Formula	Value	Use when
Accuracy	(TP + TN)/all	0.94 ("flag nothing" = 0.90)	Balanced classes, equal costs only
Precision (PPV)	TP/(TP + FP)	0.75	False alarms are expensive
Recall (sensitivity, TPR)	TP/(TP + FN)	0.60	Misses are expensive
F1 / Fβ	$2PR/(P + R)$ / $(1+\beta^2)PR/(\beta^2P + R)$	0.667	One number, imbalanced data; F2 favours recall
Specificity / FPR	TN/(TN + FP) / FP/(FP + TN)	0.978 / 0.022	Screening; ROC axes
ROC-AUC	P(random positive scored above random negative)	positives (0.9, 0.7, 0.4) vs negatives (0.8, 0.3, 0.2) → 7/9 = 0.778	Threshold-free ranking quality, moderate imbalance
PR-AUC (average precision)	Area under precision–recall curve; no-skill = base rate	—	**Rare positives** (fraud, ads, disease)
Log loss	$-\text{mean}[y \ln p + (1-y) \ln(1-p)]$	—	Probabilities feed decisions
Brier score	$\text{mean}((p - y)^2)$	y (1,0,1,1), p (0.9,0.2,0.6,0.3) → 0.175; p ≡ 0.5 → 0.25	Calibration quality (proper scoring rule)
ECE	Weighted gap between mean score and observed rate per bin	—	Calibration monitoring (ads)
Macro / weighted averages	Mean of per-class metrics (unweighted / by support)	—	Multiclass; macro exposes weak minority classes

Choosing a threshold (§6.3): with calibrated p, flag when **p > C_FP / (C_FP + C_FN)** (review $5, missed fraud $100 → p > 0.048). Fraud and moderation use **two thresholds**: approve / review or step-up / block.

Ranking and retrieval (§6.4, §13.11, §17.3)

Metric	Formula	Use when
Precision@k	relevant in top k ÷ k	Fixed number of slots
Recall@k	relevant in top k ÷ all relevant	Candidate generation (did the good items survive?)
Hit rate@k	1 if any relevant item in top k	Leave-one-out recommender tests
AP / MAP	mean of precision@i at each relevant rank ÷ #relevant; MAP = mean over queries	Binary relevance, whole list
MRR	mean of 1/rank of the first relevant result	One good answer fast (Q&A, navigation)
NDCG@k	DCG/IDCG, $DCG = \Sigma\ rel_i / \log_2(i + 1)$ (exponential variant: (2^rel − 1))	Graded relevance, position matters
Coverage, diversity, novelty	share of catalogue recommended; 1 − mean pairwise similarity; mean popularity	Beyond-accuracy health of a recommender

Worked: relevances [3, 0, 2, 1, 0, 0], one relevant grade-2 document not retrieved → DCG@5 = 4.431, NDCG@5 = **0.778**; `sklearn.metrics.ndcg_score` builds the ideal only from items you pass it and reports 0.930 — always say which NDCG. With exponential gain, [3, 0, 2] → NDCG@3 = 0.956. Recs A–E with relevant {A, C, F}: P@5 0.4, R@5 0.667, AP@5 0.556, NDCG@5 0.704.

Other families

Area	Metrics	§
Clustering (no labels)	Silhouette $s = (b - a)/\max(a, b)$ (higher), Davies–Bouldin (lower), Calinski–Harabasz (higher), inertia/BIC (compare k only) — all favour convex clusters	9.1, 9.9
Clustering (some labels)	Adjusted Rand Index (0 random, 1 perfect), NMI	9.9
Dimensionality reduction	Explained variance, reconstruction error, trustworthiness	9.5, 9.9
Anomaly detection	Precision@k, average precision on a reviewed sample	9.7
Detection / segmentation	IoU = intersection/union (worked 0.5625); mAP over IoU thresholds (VOC 0.5; COCO 0.5–0.95); Dice for masks	11.7
Association rules	support, confidence = supp(AUB)/supp(A), **lift** = confidence/supp(B) (> 1 positive)	9.8
Online / business	CTR, watch time, retention, revenue per search, resolution rate, promise-kept rate + guardrails	6.1, 16.9
Drift	$PSI = \Sigma(cur - ref)\cdot\ln(cur/ref)$: < 0.1 stable, 0.1–0.25 watch, > 0.25 investigate; KS statistic D; null rates	16.8
Fairness	Selection rate, TPR, FPR, PPV and calibration per group; disparate-impact ratio (four-fifths rule)	19.2–19.3

20.8 Validation and tuning (Ch 4, 6)

Splitter	Folds	Use for
`train_test_split(stratify=y)`	One hold-out	Final test set
`KFold(5, shuffle=True)`	Random equal folds	Regression, i.i.d. rows
`StratifiedKFold`	Same class mix in every fold	Classification (default in `cross_val_score` for classifiers)
`GroupKFold` / `StratifiedGroupKFold`	One entity per fold	Users, patients, devices
`TimeSeriesSplit(gap=…)`	Train on past, validate on the next block	Anything time-ordered; gap mimics label delay
`RepeatedStratifiedKFold` / `LeaveOneOut`	Repeats / k = n	Small data / tiny data only
Nested CV	Outer loop around the whole tuning procedure	Unbiased estimate of "tune then train"

Report CV as **mean ± std**; differences under about one std are not clearly real (§6.6). Expected test error = **bias² + variance + irreducible noise**; averaging B models with variance σ^2 and correlation ρ gives variance $\rho\sigma^2 + (1-\rho)\sigma^2/B$ — bagging shrinks the second term, random feature subsets shrink ρ (§6.7, §7.2).

Tool	Answers	Read it
Learning curve (score vs training size)	Would more data help?	Curves converge low → more capacity; big persistent gap → more data / regularisation
Validation curve (score vs one hyperparameter)	Where is the sweet spot?	Pick the peak of the validation curve

Tuning strategies (§6.9)

Method	Budget control	Best for
Grid search	Product of grid sizes (4 × 5 values × 5 folds = 100 fits)	1–2 hyperparameters, cheap models
Random search	`n_iter`	Default for 3+ hyperparameters (explores each dimension more finely)
Successive halving / Hyperband	Resource per round	Many candidates, scalable resource (rows, epochs)
Bayesian optimisation (Optuna TPE, Gaussian process + expected improvement)	Trials	Expensive trials (boosting on big data, deep nets)

Recipe: defaults + baseline → tune the few that matter (boosting: learning rate + tree size; linear/SVM: C, γ; trees: depth/leaf size) → log-uniform ranges → widen a range if the best value sits on its edge → refit on all training data → test once. **Early stopping** (§6.10): monitor validation loss, stop after `patience` rounds without improvement, restore the best weights; never early-stop on the test set.

Probability calibration (§6.5): reliability diagram + Brier/log-loss on held-out data; fix with **Platt scaling** (sigmoid on the score, small data) or **isotonic regression** (step function, roughly ≥ 1,000 examples) via `CalibratedClassifierCV` ; recheck when the base rate shifts. ROC-AUC is unchanged by any strictly increasing map.

20.9 Deep learning: layers, activations, optimizers, regularisation, debugging (Ch 10)

Building blocks (§10.1, §10.3, §10.4)

Piece	Formula / rule
Neuron	$z = w \cdot x + b$, $a = f(z)$
Dense layer	$h = f(Wx + b)$; batch: $H = f(XW^\top + b)$; params = inputs × outputs + outputs (256 → 128: 32,896)
Why nonlinearity	Without f, $W_2(W_1x + b_1) + b_2$ is still one linear map; one perceptron can't learn XOR, one hidden layer can
Forward → loss → backward → update	$\theta \leftarrow \theta - \eta \cdot \partial L/\partial\theta$; backprop = chain rule from the loss backwards, ≈ 2–3× a forward pass
Output gradient	sigmoid + BCE and softmax + CE: $\partial L/\partial \text{logits} = p - y$
Weight gradient	(layer input)$^\top$ @ (gradient at the layer's pre-activation)
Gradient check	$(L(\theta + \varepsilon) - L(\theta - \varepsilon))/2\varepsilon \approx$ analytic gradient

Activations (§10.2)

Name	Formula	Range	Use
Sigmoid	$1/(1 + e^{-z})$; $\sigma' \le 0.25$	$(0, 1)$	Binary / multi-label output; LSTM gates
Tanh	$(e^z - e^{-z})/(e^z + e^{-z})$; max slope 1	$(-1, 1)$	RNN hidden states
ReLU	$\max(0, z)$	$[0, \infty)$	**Default hidden layer**; can "die"
Leaky ReLU / PReLU / ELU	z or αz ($\alpha \approx 0.01$) / α learned / $\alpha(e^z - 1)$	—	When many ReLUs die
GELU / SiLU (Swish)	$z \cdot \Phi(z)$ / $z \cdot \sigma(z)$	—	Transformers (BERT, GPT), EfficientNet, SwiGLU in LLMs
Softmax	$e^{z_i} / \Sigma e^{z_j}$ (subtract max first)	sums to 1	One label out of K; (2, 1, 0.1) → (0.659, 0.242, 0.099)
Identity	z	$\mathbb{R}$	Regression output

Optimizers (§10.5) — g = mini-batch gradient

Optimizer	Update	Default / note
SGD	$\theta \leftarrow \theta - \eta g$	Set lr explicitly
Momentum	$v \leftarrow \mu v + g$; $\theta \leftarrow \theta - \eta v$	$\mu = 0.9$ ≈ averaging ~10 steps; Nesterov = look-ahead
AdaGrad	$s \leftarrow s + g^2$; $\theta \leftarrow \theta - \eta g/(\sqrt{s} + \varepsilon)$	Sparse features; step shrinks forever
RMSProp	$s \leftarrow \rho s + (1 - \rho)g^2$; $\theta \leftarrow \theta - \eta g/(\sqrt{s} + \varepsilon)$	PyTorch lr 0.01, α (ρ) 0.99
Adam	$m \leftarrow \beta_1 m + (1-\beta_1)g$; $s \leftarrow \beta_2 s + (1-\beta_2)g^2$; $\hat{m} = m/(1-\beta_1^t)$, $\hat{s} = s/(1-\beta_2^t)$; $\theta \leftarrow \theta - \eta\hat{m}/(\sqrt{\hat{s}} + \varepsilon)$	lr 1e-3, $\beta = (0.9, 0.999)$, ε = 1e-8
AdamW	Adam + $\theta \leftarrow \theta - \eta\lambda\theta$ (decoupled)	weight_decay 0.01; the correct "Adam with weight decay"

Schedules: step decay (÷10 at milestones), exponential, reduce-on-plateau, **cosine annealing**, **linear warmup + cosine decay** (transformers/LLMs), one-cycle. LR finder: grow lr exponentially per batch, pick ≈ one-tenth of the lr at the loss minimum.

Initialisation and gradient flow (§10.6)

Scheme	Weight variance	For
Xavier / Glorot	$2/(\text{fan_in} + \text{fan_out})$	tanh, sigmoid, linear
He / Kaiming	$2/\text{fan_in}$ (fan_in 256 → 0.0078, std 0.088)	ReLU family
LeCun	$1/\text{fan_in}$	SELU, linear

Never initialise all weights equal (symmetry never breaks). Vanishing/exploding gradients (ten sigmoid layers scale gradients by $\le 0.25^{10} \approx 9.5 \times 10^{-7}$) → ReLU, He/Xavier init, BatchNorm/LayerNorm, residual connections, gradient clipping (`clip_grad_norm_(params, 1.0)`), gates (LSTM/GRU).

Regularisation (§10.7)

Technique	How	Watch out
Dropout (p)	Zero units at random in training; survivors × 1/(1 − p) (inverted dropout)	Call `model.eval()` for inference
Weight decay	$\lambda\Sigma w^2$ or AdamW's decoupled shrinkage (1e-4…1e-2 SGD; 0.01–0.1 AdamW)	Often skip biases and norm parameters
BatchNorm	$\hat{x} = (x - \mu_{batch})/\sqrt{(\sigma^2_{batch} + \varepsilon)}$, $y = \gamma\hat{x} + \beta$; running stats at eval	Small batches; LayerNorm in transformers/RNNs, GroupNorm for small-batch vision
Early stopping	Stop after `patience` epochs without validation improvement; restore best	Report on a separate test set
Data augmentation	Label-preserving transforms (flips, crops, colour jitter, noise, Mixup/CutMix)	Training data only; don't flip a 6 into a 9
More data / smaller model	—	Often the most effective of all

Debugging checklist (§10.8)

1. Look at a few raw examples and labels **after** the data pipeline.
2. Initial loss ≈ chance: ln K for K classes (ln 10 = 2.303). Much higher → bad init/scale, wrong labels or loss.
3. **Overfit one mini-batch** to ~0 loss. Can't → a bug (labels, loss, no gradient flow, missing `optimizer.step()`).
4. LR range test; start with Adam 1e-3 (or SGD + momentum at the suggested lr).
5. Train with a validation set; plot both curves; normalise inputs with **training** statistics.
6. Only then add regularisation and tune; keep a simple baseline (logistic regression / GBDT) for "what good means".

Curve looks like	Likely cause	Try
Flat from step 0, near ln K	lr far too small; bug (frozen weights, shuffled labels, no step)	Overfit one batch; check the data pipeline
Rising, NaN, spikes	lr too high, exploding gradients, log(0)	Lower lr, clip gradients, logit-based losses
Train ↓, validation ↓ then ↑	Overfitting	Regularise, early stop, more data
Both high and close	Underfitting	Bigger model, train longer, higher lr, better features
Validation better than train	Dropout/augmentation only in training	Usually fine — check the split
Jagged	Small batch or high lr	Larger batch, lower lr

PyTorch loop (§10.9): `model.train()` → for each batch: `logits = model(xb)` ; `loss = loss_fn(logits, yb)` ; `opt.zero_grad()` ; `loss.backward()` ; `opt.step()` (then `scheduler.step()`). Evaluate under `model.eval()` + `torch.no_grad()` . Save `state_dict()` . `CrossEntropyLoss` wants `long` class indices; `BCEWithLogitsLoss` wants float targets shaped like the logits. Batch size 32–256 to start; imbalance → `weight=` / `pos_weight=` or `WeightedRandomSampler` .

20.10 CNN, RNN and attention: shapes and formulas (Ch 11, 12)

Convolutional networks (§11.1–11.2)

Quantity	Formula	Example
Output size	out = ⌊(n + 2p − k)/s⌋ + 1	n 5, k 3: s 1 p 0 → 3; s 2 → 2; p 1 → 5 ("same"); 224, k 7, s 2, p 3 → 112
Conv weights	k·k·C_in·C_out + C_out	`Conv2d(64, 128, 3)` → 73,856; `Conv2d(3, 16, 3)` → 448 vs 49,168 for a dense layer on 32×32×3
Tensor layout	PyTorch N×C×H×W; TensorFlow/NumPy images N×H×W×C	`permute(0, 3, 1, 2)`
Pooling	2×2 max, stride 2 halves H and W; no parameters; global average pooling before the head	
Receptive field	each k×k layer adds (k − 1) × product of earlier strides	stacked 3×3 (stride 1): 3, 5, 7 — three 3×3 = one 7×7 with $27C^2$ vs $49C^2$ weights
1×1 conv	mixes channels only	bottlenecks (Inception, ResNet)
Depthwise-separable	per-channel spatial conv + 1×1	≈ 8–9× less compute for 3×3 (MobileNet)
Residual block	y = ReLU(x + F(x))	identity gradient highway (ResNet)
ViT patches	(H/P)·(W/P) tokens	224×224, 16×16 → 196

Architecture timeline (§11.4): LeNet-5 (1998, conv-pool) → AlexNet (2012, ReLU + dropout + GPUs, top-5 15.3%) → VGG (2014, stacked 3×3) → Inception/GoogLeNet (2014, parallel branches, 1×1, GAP) → **ResNet** (2015, skip connections, 3.57% top-5 ensemble) → MobileNet (2017, depthwise-separable) → EfficientNet (2019, compound scaling of depth/width/resolution) → ViT (2020, patches + transformer) → ConvNeXt (2022).

Transfer learning (§11.5): small data similar to ImageNet → freeze backbone, train a new head; more or different data → fine-tune (all or top blocks) with a ~10× smaller lr; preprocess exactly like the pretraining data. **Augmentation (§11.6):** `RandomResizedCrop` , `RandomHorizontalFlip` , `ColorJitter` , `RandomErasing` , Mixup/CutMix, RandAugment — training only.

Vision tasks (§11.7): classification (label) · detection (box + class; IoU, **NMS**: keep the best box, drop overlaps above an IoU threshold, repeat; two-stage Faster R-CNN vs one-stage YOLO grid; mAP) · semantic segmentation (class per pixel; **U-Net** encoder–decoder with concatenated skip connections) · instance segmentation (mask per object; Mask R-CNN, SAM).

Recurrent networks (§12.2–12.5)

Model	Equations	Notes
RNN	$h_t = \tanh(W_{xh} x_t + W_{hh} h_{t-1} + b)$	Shared weights over time; BPTT; truncated BPTT for long sequences
LSTM	$f_t, i_t, o_t = \sigma(W[h_{t-1}, x_t] + b)$; $\check{c}_t = \tanh(\dots)$; $\mathbf{c_t = f_t \odot c_{t-1} + i_t \odot \check{c}_t}$; $h_t = o_t \odot \tanh(c_t)$	Additive cell path fixes vanishing gradients; forget-gate bias 1; 4 weight blocks
GRU	$z_t, r_t = \sigma(\dots)$; $\tilde{h}_t = \tanh(W[r_t \odot h_{t-1}, x_t])$; $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$	3 weight blocks, similar accuracy
Seq2seq	Encoder → context → decoder; teacher forcing in training; greedy / **beam search** / sampling at inference	Fixed-size bottleneck → attention

Worked LSTM unit: c = 2.0, f 0.9, i 0.2, č 0.5 → c_new = 1.9; o 0.7 → h = 0.7·tanh(1.9) = 0.669. PyTorch `nn.LSTM(input, hidden, num_layers, batch_first=True)` takes (batch, time, features) and returns (output, (h_n, c_n)). Parameter ratio RNN : GRU : LSTM = 1 : 3 : 4.

Attention and transformers (§12.6–12.7)

Idea	Formula / fact
Scaled dot-product attention	**Attention(Q, K, V) = softmax(QKᵀ/√d_k)·V** — score, softmax, weighted sum
Why √d_k	Dot products of random unit-variance vectors have std √d_k; scaling keeps softmax out of saturation
Worked example	Q = K = [[1,0],[0,1],[1,1]], V = [[1,2],[3,4],[5,6]] → query 1 weights (0.401, 0.198, 0.401), output [3.0, 4.0]
Self- vs cross-attention	Q, K, V from the same sequence / Q from decoder, K and V from encoder
Causal mask	Upper triangle set to −∞ before softmax (decoders)
Multi-head	h heads of size d_model/h, concatenated and projected
Positional encoding	PE(pos, 2i) = sin(pos/10000^(2i/d)), PE(pos, 2i+1) = cos(…); or learned / RoPE
Block	Multi-head attention and a position-wise FFN (≈ 4× wider), each wrapped in a residual connection + LayerNorm; original: 6 + 6 layers, d_model 512, 8 heads
Families	Encoder-only (BERT), decoder-only (GPT, Claude, LLaMA), encoder–decoder (T5, Whisper)
Cost	Attention O(T²·d) vs RNN O(T·d²); path length O(1) vs O(T); KV cache grows with context

Text (§12.8–12.9): normalise → tokenize → vocabulary (+ `<pad>`, `<unk>`, `<sos>`, `<eos>`) → ids → pad + mask. Subword tokenizers (BPE: merge the most frequent adjacent pair repeatedly; WordPiece; SentencePiece) have no OOV problem; ≈ 0.75 English words per token. word2vec: skip-gram (centre → context) or CBOW (context → centre) with negative sampling; GloVe, fastText (character n-grams), contextual embeddings (BERT).

Time series (§12.10): past-only features (lags, **shifted** rolling means, calendar); chronological splits (`TimeSeriesSplit`); beat naive and **seasonal-naive** baselines; ARIMA(p, d, q) = autoregression + differencing + moving average; trees can't extrapolate a trend → model differences.

20.11 Recommenders (Ch 13, 17.1)

Topic	Key facts
Framing	Fill a huge, sparse user–item matrix; optimise **top-K ranking** on an outcome closer to value than clicks (watch time, saves, purchases)
Feedback	Explicit (stars, real negatives, scarce) vs implicit (clicks/plays, abundant, unobserved ≠ negative → confidence c = 1 + α·count and sampled negatives)
Similarity	Cosine (direction), dot product (includes magnitude/popularity), adjusted cosine (subtract each user's mean), Jaccard |A∩B|/|A∪B| for sets; for unit vectors ‖a−b‖² = 2 − 2cos
Content-based vs CF	Attributes (handles new items, over-specialises) vs other users' behaviour (quality and surprise; cold start, popularity bias); hybrids win
Item-based CF	r̂(u,i) = mean(u) + Σⱼ S(i,j)(r(u,j) − mean(u)) / Σⱼ|S(i,j)|; precompute item–item neighbours (Amazon, 2003)
Matrix factorisation	r̂ = μ + b_u + b_i + p_u·q_i; train on **observed** cells with L2; SGD: err = r − r̂, p_u ← p_u + η(err·q_i − λp_u); ALS: fix one side, solve ridge per user/item (parallel); implicit ALS covers all cells with confidence weights
Embeddings	Short learned vectors; routes: MF, two-tower, graph (PinSage), pretrained content encoders; item2vec = word2vec on sessions; "near" only means near for the training objective
Two-tower	User tower and item tower meet only at a dot product → item vectors precomputed into an ANN index, user tower per request; in-batch softmax with temperature, **logQ correction**, a few hard negatives; ship tower + index as **one version**
Funnel	Retrieval 10^8 → 10^3 (recall) → ranking 10^3 → 10^2 (precision/NDCG, rich crosses; GBDT, DLRM, Wide & Deep, DCN) → re-ranking → 10–50 (diversity, freshness, policy)
MMR	next = argmax [λ·rel(j) − (1 − λ)·max$_s$ sim(j, s)]
ANN	IVF (k-means lists; `nprobe` ↑ = recall ↑), HNSW (layered graph; M, ef_search), PQ (compressed codes; IVF-PQ), trees (Annoy); always report recall vs exact search with latency
Cold start	New user: popularity, onboarding, context, fast session updates; new item: content features + controlled exploration (bandits)
Evaluation	Time-based split, full-catalogue ranking (sampled metrics mislead); P/R/hit rate@k, MAP, NDCG, coverage, diversity, novelty; **popularity baseline**; then A/B or interleaving
Bias	Popularity bias + feedback loops → measure (coverage, Gini of exposure), correct (logQ, IPS), explore, re-rank; log impressions and positions

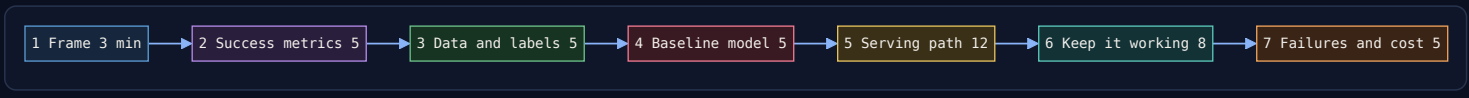
---

20.12 Reinforcement-learning update rules (Ch 14)

Idea	Formula	Notes / worked value
MDP	(S, A, P(s'|s,a), R, γ); Markov: next state depends only on current state and action	§14.2
Return	$G_t = \Sigma_k\ \gamma^k\ r_{t+k+1} = r_{t+1} + \gamma G_{t+1}$	γ = 0.9: horizon ≈ 1/(1−γ) = 10; $0.9^{10} \approx 0.35$
Value functions	V^π(s) = E[G | s]; Q^π(s,a) = E[G | s, a]; π*(s) = argmax_a Q*(s,a)	Q needs no model to act
Bellman expectation	V^π(s) = $\Sigma_a\ \pi(a\|s)\ \Sigma_{s'}\ P(s'\|s,a)[R + \gamma V^\pi(s')]$	Policy evaluation
Bellman optimality	V*(s) = $\max_a\ \Sigma_{s'}\ P[R + \gamma V^*(s')]$; Q*(s,a) = $\Sigma_{s'}\ P[R + \gamma\ \max_{a'}\ Q^*(s',a')]$	Treasure grid: 10, 8, 6.2, 4.58, …, −0.43 at the start
ε-greedy	random arm with probability ε, else best	Decay ε
UCB1	$\text{argmax}_a\ [Q(a) + c\sqrt{(\ln t\ /\ N_a)}]$, c = √2 in theory	Tune c for your horizon
Thompson sampling	Sample $\theta_a \sim \text{Beta}(\text{wins} + 1, \text{losses} + 1)$, pull argmax	Probability matching
Value iteration	V(s) ← $\max_a\ \Sigma\ P[R + \gamma V(s')]$ until stable	Needs the model (DP)
Policy iteration	Evaluate π, make π greedy on V, repeat	Few rounds
Monte Carlo	V(s) ← V(s) + α[G_t − V(s)]	Unbiased, high variance, end of episode
TD(0)	V(s) ← V(s) + α[r + γV(s') − V(s)]; **δ = r + γV(s') − V(s)**	Bootstraps, lower variance, every step
Q-learning (off-policy)	Q(s,a) ← Q(s,a) + α[r + γ $\max_{a'}$ Q(s',a') − Q(s,a)]	Learns the greedy policy; cliff edge path
SARSA (on-policy)	Q(s,a) ← Q(s,a) + α[r + γQ(s',a') − Q(s,a)], a' = next action taken	Learns what it does; safer path
DQN	Minimise Huber(r + γ $\max_{a'}$ Q(s',a'; θ^-) − Q(s,a; θ)) on replay mini-batches	Experience replay + target network; Double DQN: select with online, evaluate with target
REINFORCE	$\theta \leftarrow \theta + \alpha \cdot G_t \cdot \nabla \log \pi(a_t\|s_t)$	High variance → baseline: $A_t = G_t - V(s_t)$
Actor-critic	actor: θ ← θ + α·δ·∇log π; critic: TD learning on V	PPO = clipped actor-critic update
RLHF	SFT → reward model (−ln σ(s_w − s_l)) → PPO on reward − β·KL(π ‖ π_SFT)	DPO skips the RL loop; GRPO uses group-mean baselines

Pitfalls (§14.12): reward hacking (a re-collectable +3 per step is worth 3/(1 − 0.9) = 30), sample inefficiency, sparse rewards, credit assignment, seed instability, sim-to-real gap, unsafe exploration, the **deadly triad** (function approximation + bootstrapping + off-policy).

20.13 ML system design: the 7-step framework and napkin math (Ch 15, 17, 18)



Step	Say out loud	§
1. Frame	Family (predictive / generative / agentic — does it output a score, content or an action?); the known system it resembles; scope; ML task: input, output, category, **ML objective**	15.2, 15.3
2. Success	Online business metric + offline ML metric + guardrails; latency (p95/p99); scale (QPS, peak); cost per request or per completed task	15.3
3. Data	Sources; **label source and delay**; freshness (batch / streaming / request-time); feature groups; leakage and privacy	15.3, 16.3, 16.4
4. Model	Baseline (popularity, rules, LR) → GBDT → deep (two-tower, multi-task); name the trigger to upgrade	15.3
5. Serving	What's offline vs per request; caches (with permissions); cheaper paths; timeouts and fallbacks; funnel if the catalogue is big	15.3, 16.5, 16.6
6. Keep it working	Log inputs/outputs/versions → join outcomes → monitor 4 layers → gated retraining → shadow → canary → ramp	15.3, 16.8, 16.9
7. Failures and cost	Specific failure list with controls; cost = requests × share × calls × price, in the same unit as the benefit	15.3

Business goal → ML objective (§15.3): time spent → P(watch ≥ X s) or expected watch time (not raw clicks) · fraud losses → P(fraud) + cost-based thresholds · delivery promise → minutes to delivery + a product-owned buffer · search revenue → P(purchase | query, item) · churn → P(cancel within 30 days) (label is 30 days late).

Four assumptions that break with a model (§15.1): correctness is a spectrum (need quality metrics) · behaviour depends on data (drift) · compute is not cheap (model calls cost money) · failures are silent (a wrong answer is HTTP 200).

Napkin numbers (illustrative, from the chapters)

Quantity	Arithmetic	Result	§
QPS (queries per second)	requests/day ÷ 86,400; peak ≈ 2–3×	10 M/day → 116 avg, ~347 peak (3×)	15.3
Homepage recs	100 M DAU (daily active users) × 5 loads ÷ 86,400	≈ 5,800 avg, ~17 K peak (3×); × 500 candidates ≈ 8.7 M scores/s	17.1
Feed ranking	500 M DAU × 5 sessions × 3 calls ÷ 86,400	≈ 86.8 K/s; × 500 posts ≈ 43 M scores/s	17.2
Ads scoring	100 K requests/s × 300 candidates	30 M scores/s in a 50–100 ms auction	18.1
Vector memory	N × dims × 4 bytes (float32)	50 M × 128 → 25.6 GB; 100 M × 512 → 204.8 GB; PQ to 64 B → 6.4 GB	17.1, 18.6
Batch precompute	users × items per user	25 M × 500 = 12.5 B rows; active 5 M only → 2.5 B	16.5
LLM cost	10 M req/day × $0.01	$100 K/day ≈ $3 M/month	15.1
Support bot	500 K orders × 8% contacts × 10 calls × $0.01	40 K conversations, $4 K/day; compare **per resolved issue**	15.4
Moderation cascade	200 M uploads/day, 1% flagged to heavy models, 0.1% to humans	2 M heavy/day (≈ 23/s), 200 K human decisions/day (~670 reviewer-days)	18.3
A/B sample size	$n \propto 1/\delta^2$	5% CTR, +1% relative lift → ~3.0 M users per arm; half the lift → ~4× users	16.9
Latency budget (small model)	inference 1–10 ms, online-store lookup 2–10 ms, inline features 0–15 ms, routing/auth 1–5 ms, logging 1–2 ms	batch KV lookup ~1 ms vs online 10–100 ms	16.5, 16.6
Fraud	10 K tx/s × 0.2% fraud; ~50 ms authorisation, ~10 ms for the model	~20 fraud attempts/s	18.2
Video indexing	1 M uploads/day × 600 s ÷ 2 s per sampled frame	300 M frame encodings/day ≈ 3,500/s — indexing, not querying, dominates	17.4
Blurring batch	5 M panoramas × 24 tiles × 10 ms	1.2 M GPU-s/day ≈ 14 GPUs busy all day	18.7

Thirteen case studies: the one-row-per-design comparison (task type, features, model, metrics, hardest challenge) is the table at the end of Chapter 18.

20.14 Production and MLOps checklist (Ch 16)

Architecture

- ☐ Five pieces named: training pipeline, feature layer, inference path, serving platform, monitoring loop; offline plane vs online plane (§16.1).
- ☐ Training pipeline: data preparation with **point-in-time-correct joins** → training → evaluation vs production → model registry with lineage (§16.2).
- ☐ One feature definition for training and serving; serving features logged and reused for training (Rules of ML — Google's published list of ML-engineering rules — #29, #32, #37) (§16.2).
- ☐ Feature store: offline history for training, online latest values for serving; batch vs streaming vs request-time features with freshness alerts (§16.4).
- ☐ Inference mode chosen by the deciding question "does it need request-time information?": batch, online, streaming, or precompute-then-rerank (§16.5).

Serving

- ☐ Stateless, versioned model service; itemised latency budget; batched feature reads; hot-entity caches (§16.6; caches, load balancers and timeouts are explained in System Design Fundamentals).
- ☐ Timeouts and an agreed **fallback** (previous model, cached score, heuristic); fail-open vs fail-closed decided per risk (§16.2, §16.6).
- ☐ CPU benchmark before GPUs; distillation / quantisation / pruning if needed (§16.6).
- ☐ Model version logged with every prediction (§16.6).

Evaluation and release

- ☐ Offline metric matches the problem (PR-AUC + threshold, recall@k + NDCG, MAE/RMSE + p90 + cohorts), calibration if scores are used as probabilities, naive and production baselines (§16.7).
- ☐ Release ladder: offline gate (overall **and per segment**) → shadow → sticky canary → ramp with a pre-agreed abort rule; rehearsed rollback of model + features + config (§16.6).
- ☐ A/B test: randomisation unit, one primary metric, guardrails, power (MDE — minimum detectable effect, α, $1 - \beta$), whole weeks, no peeking (or a sequential test), sample-ratio-mismatch check (did each arm get the share of users it should?); interleaving / switchback / holdback when appropriate (§16.9).

Monitoring and retraining

- ☐ Four layers: service health → input distributions (PSI, KS, **null rates**) → prediction distribution → outcomes as labels arrive (§16.8).
- ☐ Data drift vs label shift vs concept drift vs plain bugs; alerts start investigations (§16.8).

- ☐ Retraining cadence set by measured decay and label maturity; triggered retrains go through the same gates; never auto-promote (§16.8).
- ☐ Feedback loops: log impressions and selection probabilities; small risk-approved exploration (§16.8).

MLOps (§16.10)

- ☐ Experiment tracking (code version, params, **data hash**, metrics, artifacts) — MLflow, W&B.
- ☐ Data and model versioning (DVC, lakeFS, Delta), data validation that fails the pipeline (TFDV, Great Expectations, Pandera).
- ☐ Registry with stages and owners; orchestration (Airflow, Kubeflow, Dagster); CI with feature unit tests and training–serving consistency tests; CD with shadow/canary/auto-rollback; continuous training (MLOps level 1 → 2).
- ☐ Test the infrastructure independently from the ML: push a trivial model through, confirm identical offline and online predictions (Rules of ML #5).
- ☐ Labels: source, delay, cost; downsample-then-recalibrate; temporal splits; PII minimised, deletion path defined (§16.3).

20.15 Responsible-ML checklist (Ch 19)

Run the pre-launch checklist in §19.11 (framing → data → model and evaluation → deployment and monitoring). Specifics to remember: pick the criterion by harm (demographic parity, equal opportunity, equalized odds, calibration) with legal input; audit per group **and intersection** with CIs, disparate-impact ratio < 0.8 as a warning, proxy test (predict the group from features); reweighing $w(g, y) = P(g)P(y)/P(g, y)$, fairness penalty, or group thresholds fitted on validation data, then re-audit; SHAP values add up to $f(x) - E[f]$, never causal; human oversight and appeal (GDPR Art. 22); authentication, rate limits, label-only outputs.

Key formulas: disparate impact = $P(\hat{Y}=1 \mid B)/P(\hat{Y}=1 \mid A)$ (0.35/0.50 = 0.70) · Chouldechova: $FPR = p/(1-p)\cdot(1-PPV)/PPV\cdot(1-FNR)$ · ε-DP: $P[M(D) \in O] \le e^{\varepsilon}\cdot P[M(D') \in O]$; Laplace noise scale $\Delta f/\varepsilon$ (count, ε 0.5 → 2); k queries at ε each cost $k\varepsilon$ · FGSM: $x_{adv} = \text{clip}(x + \varepsilon\cdot\text{sign}(\nabla_x L), 0, 1)$ · FedAvg = data-size-weighted average of client weights.

20.16 Decision tables: which algorithm, which metric

Which algorithm for which problem

Problem	Start with (baseline)	Then try	Avoid / notes	§
Tabular classification or regression, < ~1 M rows	Logistic / linear regression; majority-class or mean baseline	Random forest → **gradient boosting** (LightGBM, XGBoost, CatBoost, `HistGradientBoosting`)	Deep nets usually lose on small tables; k-NN beyond ~20–50 features	7.6, 8.4
Many categorical features	CatBoost or `HistGradientBoosting` with native categoricals	Target encoding (cross-fitted) + GBDT	One-hot of 100 K levels for a linear model	4.4, 7.4
High-dimensional sparse text	TF-IDF + logistic regression / linear SVM; Naive Bayes for tiny data	Fine-tuned transformer if data and budget allow	RBF SVM on sparse text	4.6, 8.2, 8.3
Images	Fine-tune a pretrained CNN/ViT (ResNet-50, EfficientNet) with augmentation	Bigger backbone, more data	Training from scratch on a small dataset	11.5
Detection / segmentation	YOLO-style one-stage (real time) or Faster R-CNN; U-Net for masks	Transformer detectors (DETR)	Judging detectors by accuracy (use mAP)	11.7
Sequences / text generation	Transformer (pretrained)	LSTM/GRU for small, streaming or on-device models	Simple RNN for long dependencies	12.4, 12.7
Time-series forecast	Naive and seasonal-naive; ETS / ARIMA for one series	Global GBDT on lag features (model differences); deep models with lots of data	Random splits; trees on trending levels	12.10
Clustering, round groups	k-means (k-means++, scaled, silhouette/elbow for k)	GMM (ellipses, soft), agglomerative (hierarchy)	k-means on moons/rings	9.1–9.4
Clustering, odd shapes / noise	DBSCAN / HDBSCAN	—	One eps for mixed densities (use HDBSCAN)	9.3
Compression / pre-step	PCA (standardise first; ~95% variance)	Kernel PCA, TruncatedSVD for sparse	PCA for visualising non-linear structure	9.5
2-D visualisation	PCA first, then t-SNE / UMAP	UMAP for big data or new points	Reading cluster sizes/gaps in t-SNE	9.6
Anomaly detection, unlabelled	Robust z / IQR per feature; Isolation Forest multivariate	LOF (mixed densities), One-Class SVM (clean novelty data), autoencoders	Plain z-scores on tiny or skewed samples	9.7
Rare-event classification (fraud)	Rules + logistic regression → GBDT with velocity and graph features	Graph neural nets for rings, anomaly scores	Accuracy; single threshold	18.2
Recommendations	Popularity + item-based CF / co-occurrence	Implicit MF (ALS) → two-tower retrieval + GBDT/deep ranker	Pure CF for always-new catalogues	13.4–13.8
Search ranking	BM25 + popularity	Hybrid lexical + semantic retrieval, LambdaMART / deep L2, cross-encoder top-k	Training on raw position-biased clicks	17.3
Choosing among a few options with live feedback	A/B test	Bandits (Thompson, UCB, contextual LinUCB)	Fixed ε forever	14.4
Sequential decisions with a simulator	Value iteration (known model) / Q-learning (small)	DQN (discrete), PPO/SAC (continuous)	Full online RL on live money or people	14.5–14.9
Must explain every decision	Linear/logistic model, small tree, rule list, GAM	GBDT + SHAP reason codes	Deep black box without oversight	19.5

Which metric for which problem

Situation	Primary metric	Also report	Don't	§
Balanced classes, equal costs	Accuracy + confusion matrix	F1, log loss	—	6.3
False positives costly (spam hiding real mail)	Precision at a recall floor	PR curve	Quote precision without threshold and base rate	6.3
Misses costly (screening, fraud)	Recall at a precision floor	Expected cost	—	6.3
Rare positives (≤ ~1%)	PR-AUC / average precision + P/R at the operating threshold	Base rate, expected $ loss	Accuracy; ROC-AUC alone	6.3, 16.7
Comparing rankers, moderate imbalance	ROC-AUC	PR-AUC	—	6.3
Probabilities used in arithmetic (auctions, expected cost)	Log loss, Brier, ECE per segment	Reliability diagram	Trust AUC for calibration	6.5, 18.1
Multiclass with weak minority classes	Macro-F1 / per-class P, R	`classification_report`	Micro-average only	6.3
Regression, errors cost linearly	MAE	p90 error, per cohort	—	6.2
Regression, big misses very costly	RMSE	MAE, p90, per cohort	R^2 alone	6.2
Retrieval / candidate generation	Recall@k	Coverage	NDCG at this stage	6.4, 13.8
Ranked list with graded relevance	NDCG@k (state the gain and ideal)	MAP, MRR	Sampled-negative metrics	6.4, 17.3
One right answer fast	MRR	Hit rate@k	—	6.4
Recommender health	NDCG/MAP + coverage, diversity, novelty	Popularity baseline	Accuracy alone	13.11
Clustering without labels	Silhouette / DB / CH to compare settings	Stability, downstream value	Trust them on non-convex shapes	9.9
Object detection	mAP at IoU thresholds	Per-class AP	Accuracy	11.7
Moderation	Prevalence of violating views per category	Wrongful removals; share of appeals won	Items removed	18.3
Launch decision	Online business metric in an A/B test	Guardrails (latency, complaints, fairness)	Offline metric alone	16.9
Fairness	The criterion that matches the harm (equal opportunity, equalized odds, parity, calibration)	All others + group sizes and CIs	One unnamed "fairness" number	19.2

20.17 Top 50 interview questions

Crisp answers; each points to the full treatment.

Foundations

1. **What's the difference between AI, ML and deep learning?** → AI is the goal of intelligent behaviour (including rules and search); ML learns behaviour from data; deep learning is ML with many-layer neural networks that learn their own features. (§1.1)
2. **Supervised vs unsupervised vs self-supervised vs reinforcement?** → Labels / no labels / labels generated from the data (next token) / rewards from actions in an environment. (§1.4)
3. **Parameters vs hyperparameters?** → Parameters are learned in training (weights); hyperparameters are set before it (learning rate, depth, C) and tuned on validation data. (§1.6)
4. **Why three data splits?** → Train fits parameters, validation chooses hyperparameters and models, the test set is used once so its score stays unbiased. (§1.6, §4.7)
5. **What is data leakage? Give examples.** → Information unavailable at prediction time reaching training or evaluation: post-outcome features, scaling or feature selection fitted before splitting, random splits of time series, duplicates across splits. Prevent with point-in-time features, split-first pipelines and time/group splits. (§1.10, §4.8)
6. **Explain the bias–variance trade-off.** → Test error = bias² + variance + noise. Simple models underfit (high bias), flexible ones overfit (high variance); total error is U-shaped in complexity. (§6.7)
7. **How do you detect and fix overfitting?** → Train score much better than validation; fix with more data, regularisation, a simpler model, early stopping, bagging or augmentation. (§1.10, §6.7)
8. **Explain gradient descent and the learning rate.** → Repeat $w \leftarrow w - \eta \nabla L$. Too small is slow, too large diverges; mini-batches are the practical default; scale features for a rounder loss surface. (§2.5, §5.4)
9. **Explain Bayes' theorem with an example.** → Posterior ∝ likelihood × prior. A 90%-sensitive, 5%-false-positive test for a 1%-prevalence disease gives only ~15% P(sick | positive) — the base-rate effect behind low precision on rare events. (§2.7)
10. **What is a p-value — and what isn't it?** → P(data at least this extreme | H_0). It is not P(H_0 is true); significance ≠ importance; correct for multiple tests and don't peek. (§2.11)
11. **Correlation vs causation?** → Correlation can come from confounders, reverse causation or chance; causal claims need randomised experiments. Models find correlations — fine for prediction, risky for decisions. (§2.9)
12. **What is cross-entropy and why is it the classification loss?** → −ln of the probability given to the true class; it's the maximum-likelihood loss, convex for logistic regression, and punishes confident mistakes hardest, with gradient p − y at the logits. (§2.12, §10.3)

Data

13. **How do you handle missing values?** → Ask why they're missing (MCAR/MAR/MNAR); median or "missing" category plus a missing indicator as a default; KNN/iterative imputation when other columns predict it; fit imputers inside the pipeline; measure by masking known values. (§4.3)
14. **One-hot vs target encoding vs hashing?** → One-hot for small nominal sets; target encoding for high cardinality (cross-fitted and smoothed to avoid leakage); hashing for unbounded IDs (collisions); embeddings in deep models. (§4.4)
15. **Which models need feature scaling?** → Anything using distances, gradients or penalties: k-NN, k-means, SVM, linear/logistic with regularisation, neural nets, PCA. Trees don't. Fit the scaler on training data only. (§4.5)
16. **How do you deal with class imbalance?** → Change the metric (precision/recall, PR-AUC); move the threshold; class weights or downsampling (then recalibrate: $p \cdot w/(p \cdot w + 1 - p)$); SMOTE sometimes; resample training data only. (§4.9)
17. **Where do labels come from, and what goes wrong?** → Outcomes (late), behaviour (biased by exposure and position), human annotation (costly, needs agreement checks), weak supervision and LLMs (correlated errors). (§4.1, §16.3)

Classic models

18. **Linear vs logistic regression?** → Linear predicts a number by minimising MSE; logistic predicts a probability through a sigmoid by minimising log loss; its boundary is $w \cdot x + b = 0$ and e^w is an odds ratio. (§5.2, §5.8)
19. **Normal equation vs gradient descent?** → Exact $\theta = (X^T X)^{-1} X^T y$, $O(d^3)$, no tuning — fine for small d; GD scales to huge n and d and any differentiable loss but needs a learning rate and scaled features. (§5.3)
20. **Ridge vs Lasso?** → L2 shrinks all weights and shares weight among correlated features; L1 sets some weights exactly to zero (feature selection). ElasticNet mixes both. More regularisation → more bias, less variance. (§5.7)
21. **How does a decision tree choose a split?** → It tries every feature and threshold and keeps the split with the largest impurity decrease (Gini or entropy; variance for regression); greedy, then prune by depth/leaf size or ccp_alpha. (§7.1)
22. **Bagging vs boosting?** → Bagging trains deep trees in parallel on bootstrap samples and averages (cuts variance); boosting adds shallow trees sequentially, each fitting the current errors (cuts bias). (§7.2–7.4)
23. **Random forest vs gradient boosting?** → RF: robust, little tuning, doesn't overfit by adding trees, free OOB score. GBDT: usually the most accurate on tables but needs a small learning rate and early stopping. (§7.3, §7.4)
24. **Why can feature importance mislead?** → Impurity importance is computed on training data, favours high-cardinality features and splits credit among correlated ones; use permutation importance on held-out data or SHAP. (§7.3, §19.5)
25. **Explain SVMs and the kernel trick.** → Maximise the margin $2/\|w\|$ (C trades width for violations); the dual needs only dot products, so a kernel $K(x, z) = \varphi(x) \cdot \varphi(z)$ gives non-linear boundaries without building φ. Tune C and γ together. (§8.2)
26. **Why is Naive Bayes "naive", and why does it still work?** → It assumes features are independent given the class; classification only needs the argmax right, so it works well for text, but its probabilities are over-confident. (§8.3)
27. **How does k-NN work and when does it fail?** → Vote/average of the k closest scaled points; prediction is O(nd) and distances concentrate in high dimensions (curse of dimensionality). (§8.1)
28. **How does k-means work and how do you choose k?** → Assign to nearest centroid, recompute means, repeat (local optimum; k-means++ and restarts). Choose k by silhouette, elbow and domain sense; it assumes round, similar clusters. (§9.1)
29. **What does PCA do?** → Rotates standardised data onto orthogonal directions of maximum variance (eigenvectors of the covariance / SVD); keep enough components for e.g. 95% variance. Linear and unsupervised. (§9.5)

Evaluation

30. **Precision vs recall — which do you optimise?** → Depends on error costs: precision when false alarms hurt, recall when misses hurt; set the threshold where $p > C_{FP}/(C_{FP} + C_{FN})$ with calibrated scores, or use two thresholds with a review band. (§6.3)
31. **ROC-AUC vs PR-AUC?** → ROC-AUC = P(a positive outranks a negative), insensitive to base rate; with rare positives, FPR's huge denominator hides false alarms, so PR-AUC (no-skill = base rate) is the honest metric. (§6.3, §16.7)
32. **What is calibration and when does it matter?** → Predicted probabilities matching observed frequencies — separate from ranking. Needed when scores feed arithmetic (auctions, expected cost, human-read risk scores). Fix with Platt or isotonic on held-out data. (§6.5, §18.1)
33. **Explain k-fold cross-validation and its variants.** → k train/validate rounds averaged (mean ± std); stratified for classes, group folds for entities, forward-in-time splits with a gap for temporal data; nested CV for an unbiased tuned estimate. (§6.6)
34. **Grid vs random search vs Bayesian optimisation?** → Grid for 1–2 parameters; random search explores each dimension better on the same budget; Bayesian (Optuna) models score vs hyperparameters for expensive trials; log scales; test once. (§6.9)
35. **Explain NDCG.** → DCG sums graded relevance discounted by $\log_2(\text{rank} + 1)$; dividing by the ideal DCG normalises to [0, 1]. Rewards putting the most relevant items at the top. (§6.4, §17.3)

Deep learning

36. **How does backpropagation work?** → The chain rule applied from the loss backwards through the computational graph, reusing intermediate values, giving every gradient for about the cost of an extra forward pass; an optimizer then updates the weights. (§10.4)
37. **Why do we need non-linear activations, and which ones?** → Without them any stack is one linear map. ReLU for hidden layers (GELU/SiLU in transformers); sigmoid/softmax/identity at the output by task. (§10.2)
38. **Vanishing and exploding gradients — causes and fixes?** → Gradients are products of per-layer (or per-time-step) factors. Fix with ReLU, He/Xavier initialisation, normalisation, residual connections, gating (LSTM/GRU) and gradient clipping. (§10.6, §12.3)
39. **SGD vs momentum vs Adam vs AdamW?** → Momentum accumulates velocity; RMSProp/Adam scale each parameter's step by its recent gradient size (Adam adds bias correction); AdamW decouples weight decay. Adam(W) is the default for transformers; SGD + momentum stays strong for CNNs. (§10.5)
40. **What do dropout and batch normalisation do?** → Dropout zeroes random units in training so the net can't rely on co-adapted features (off at eval); BatchNorm normalises activations per mini-batch, allowing higher learning rates and adding mild regularisation (running stats at eval). (§10.7)
41. **Why CNNs for images?** → Weight sharing and local connectivity give far fewer parameters and translation equivariance; stacked 3×3 convs with downsampling grow the receptive field; ResNet skip connections make very deep nets trainable. (§11.1, §11.4)
42. **What is transfer learning?** → Start from a model pretrained on a big dataset, replace the head, freeze then fine-tune with a small learning rate; biggest gains when labels are scarce. (§11.5)

43. **How does an LSTM fix the RNN's problems?** → An additive cell state $c_t = f \odot c_{t-1} + i \odot \tilde{c}_t$ controlled by sigmoid gates lets information and gradients flow across many steps. (§12.4)
44. **Explain attention and the transformer.** → $\text{softmax}(QK^T/\sqrt{d_k})V$ is a learned weighted lookup; transformers stack multi-head self-attention and feed-forward layers with residuals and LayerNorm plus positional encodings — parallel over the sequence, O(1) path between tokens, $O(n^2)$ cost. (§12.6, §12.7)

Recommenders, RL and systems

45. **Content-based vs collaborative filtering, and cold start?** → Attributes vs co-behaviour; CF finds quality and surprise but fails on new users/items. Cold start: popularity, onboarding, content features and controlled exploration. (§13.4, §13.10)
46. **How do you retrieve candidates from 100 M items in milliseconds?** → Two-tower model: item vectors precomputed into an ANN index (HNSW/IVF), user tower per request; in-batch negatives with logQ correction; tower and index shipped as one version; then a ranker. (§13.7–13.9)
47. **Q-learning vs SARSA? Exploration vs exploitation?** → Q-learning bootstraps from the max next action (off-policy, learns the optimal policy); SARSA from the action actually taken (on-policy, safer while exploring). Exploration: ε-greedy, UCB, Thompson sampling. (§14.4, §14.7)
48. **Walk me through designing an ML system.** → Frame → success metrics → data and labels → baseline → serving path → keep-it-working loop → failures and cost, translating the business goal into an observable ML objective and keeping product knobs (thresholds, buffers) outside the model. (§15.3)
49. **How do you monitor a model in production, and when do you retrain?** → Health → input drift (PSI/KS/nulls) → prediction drift → outcomes; alerts trigger investigation (often a broken pipeline); retrain on measured decay through offline, shadow and canary gates. (§16.8)
50. **Your model is 94% accurate — is it fair?** → Unknown until you slice by group: selection rate, TPR, FPR, PPV and calibration per group, proxy checks; criteria conflict when base rates differ, so choose by harm and mitigate (reweighing, penalties, thresholds). (§19.2–19.4, §19.12)

20.18 Glossary (A–Z)

911 terms. Each entry: **term** — plain meaning — the section that teaches it (the link opens that chapter).

Term	Meaning	Where
1×1 convolution	Convolution mixing channels at each pixel; used to shrink or expand channel count	11.1
A2C / A3C / SAC / DDPG / TD3	Common deep actor-critic algorithms (synchronous, asynchronous, entropy-regularised, deterministic for continuous actions)	14.9
Ablation	Removing one component or feature group and retraining to measure its contribution	18.8
A/B test	Randomised experiment splitting units between control and treatment to measure a business metric	16.9
Accuracy	Share of predictions that are correct, (TP + TN)/all; misleading on imbalanced data	6.3
Acquisition function	Rule in Bayesian optimisation deciding which setting to try next (e.g. expected improvement)	6.9
Action ladder	Graded moderation responses (remove, review, reduce reach/age-gate/warn, allow) set per category and region	18.3
Activation function	Non-linear function applied to a neuron's pre-activation (ReLU, sigmoid, tanh, GELU)	10.2
Active learning	Routing the examples with the most model uncertainty (or annotator disagreement) to human labellers first	4.1
Actor-critic	Policy-gradient method where a critic's TD error δ serves as the advantage for the actor's update	14.9
AdaBoost	Boosting by re-weighting misclassified samples; each stump gets a vote α = ½ ln((1 − ε)/ε)	7.4
AdaGrad	Optimizer dividing each step by the root of the summed squared past gradients	10.5
Adam	Optimizer combining momentum and RMSProp-style per-parameter step sizes, with bias correction	10.5
Adamic-Adar	Link-prediction score summing 1/log(degree) over shared neighbours, so rare mutual friends count more	18.4
AdamW	Adam with decoupled weight decay applied directly to the weights	10.5
Adjusted cosine	Cosine similarity after subtracting each user's mean rating	13.3
Adjusted Rand Index (ARI)	Agreement between two clusterings corrected for chance: about 0 random, 1 perfect	9.9
Advantage	How much better an action is than the state's average, A = Q − V	14.8
Adversarial drift	Data changing because attackers adapt to your defences	18.2
Adversarial example	Input perturbed slightly, on purpose, so that a model misclassifies it	19.7
Adversarial training	Including adversarial examples in each training batch to gain robustness	19.7
Agent / environment	The learner that acts / the world it acts in and gets rewards from	14.1
Agentic AI	Systems that plan and take multi-step actions toward a goal using tools	1.1
Agglomerative clustering	Bottom-up clustering that repeatedly merges the two closest clusters	9.2
Aggregation / evaluation / deployment bias	One model for groups that differ / an unrepresentative benchmark / use outside the intended setting	19.1
AI (artificial intelligence)	The broad goal of making machines behave intelligently, by rules, search or learning; ML is one part of it	1.1
AlphaGo / AlphaZero	DeepMind systems combining deep RL and tree search that beat top players at Go, chess and shogi	14.11
ALS (alternating least squares)	Matrix-factorisation solver that fixes one side and solves a ridge regression for the other, alternately	13.5
Anchor boxes	Preset box shapes a detector adjusts instead of predicting boxes from scratch	11.7
ANN (approximate nearest neighbour)	Search that returns almost the exact nearest vectors while scanning only a fraction of them	13.9
Anomaly detection	Finding rare points that differ markedly from the majority, usually without labels	9.7
Anonymisation	Irreversibly removing the ability to identify people; removing names is not enough	19.6
API (application programming interface)	Defined way for one program to call another, e.g. sending a request to a model service and reading the answer	1.1
Apriori	Frequent-itemset algorithm that prunes candidates because every subset of a frequent set must be frequent	9.8
ARIMA	Time-series model: autoregression on p lags, d differences, moving average on q past errors	12.10
Article 22 (GDPR)	Right not to be subject to solely automated decisions with legal or similarly significant effects	19.9
Artifact (ML)	Saved output of a training run, such as a model file, index or preprocessing pipeline	13.7
Atomic swap (promotion)	Switching model and index versions together in one step so no request sees a mix	13.7
Attention	Weighted average of value vectors with weights softmax(q·k/√d_k) from query–key similarity	12.6
Attribution window	Period after a click during which a conversion is credited to it	18.1
AUC (ROC-AUC)	Probability a random positive is scored above a random negative; 0.5 random, 1 perfect	6.3
Autoencoder	Network trained to reconstruct its input through a narrow middle layer; large reconstruction error suggests an anomaly	9.7

Term	Meaning	Where
Autograd / automatic differentiation	Software computing exact derivatives by applying the chain rule to recorded tensor operations; PyTorch runs backprop on `.backward()`	10.4
Automatic speech recognition (ASR)	Converting speech audio into a (timestamped) transcript, e.g. to make videos searchable	17.4
Average pooling	Downsampling by averaging each window	11.2
Average precision (AP)	Area under the precision–recall curve, the usual PR-AUC; no-skill value equals the base rate	6.3
Backbone	Pretrained feature-extracting part of a network, reused under new heads	11.2
Backdoor (trojan)	Hidden trigger planted through poisoned training data that makes the model misbehave on command	19.7
Backfill	Computing a new feature or table for past dates	16.4
Backpressure	Slowing producers when consumers can't keep up, so queues don't overflow	16.5
Backpropagation (backward pass)	Chain rule applied from the loss backwards to get every parameter's gradient efficiently	10.4
Backpropagation through time (BPTT)	Backprop through an RNN unrolled over the sequence; gradients summed over time steps	12.2
Backtest	Replaying a model or policy on past data whose outcomes are already known	18.2
Bagging	Training copies of a model on bootstrap samples and averaging them to cut variance	7.3
Bag-of-words	Text representation counting each word's occurrences and ignoring order	4.6
Bandit (multi-armed)	One-state RL problem of choosing among arms with unknown payoffs; isolates exploration vs exploitation	14.4
Baseline	Score of a trivial predictor (majority class, mean, last value, popularity) every model must beat	6.1
Base rate	How often the positive class occurs in the population; drives precision and PR-AUC	6.3
Base-rate fallacy	Ignoring how rare something is, e.g. trusting a positive test for a rare disease too much	2.7
Basis point (bp)	One hundredth of a percent (0.01%)	18.2
Batch inference	Predictions precomputed on a schedule and stored so a request becomes a lookup	16.5
Batch learning	Training on the full dataset at once and redeploying a frozen model periodically	1.7
Batch normalisation (BatchNorm, BN)	Layer normalising activations over the mini-batch, then scaling and shifting with learned γ and β	10.7
Bayesian optimisation	Tuning with a surrogate model of score vs hyperparameters and an acquisition function	6.9
Bayes' theorem	$P(A\|B) = P(B\|A)\cdot P(A)/P(B)$: posterior ∝ likelihood × prior	2.7
Beam search	Decoding that keeps the k most probable partial sequences at each step	12.5
Bellman equation	Value now = expected reward now + γ × value of the next state	14.3
Bernoulli distribution	Single yes/no trial that succeeds with probability p	2.6
BERT / GPT	Landmark encoder-only (understanding) and decoder-only (generation) transformer families	12.7
Bias (fairness)	Systematic, unjustified difference in how a model treats or performs for groups	19.1
Bias mitigation (pre / in / post-processing)	Fixing the data / the training objective / the decision rule to improve a fairness criterion	19.4
Bias (statistical)	Error from wrong assumptions; distance of the average model from the truth	6.7
Bias–variance trade-off	Simple models miss patterns (bias), flexible ones chase noise (variance); the best model balances the two	1.10
BIC	Bayesian Information Criterion: likelihood penalised by parameter count, lower is better (choose k in GMMs)	9.4
Bidirectional RNN	Two RNNs reading the sequence forwards and backwards, outputs combined	12.2
Binary classification	Predicting one of two classes (yes/no, fraud/legit)	5.8
Binary cross-entropy (BCE)	Log loss for 0/1 targets with sigmoid outputs	10.3
Binomial distribution	Number of successes in n independent yes/no trials	2.6
BM25	Lexical relevance score rewarding rare query terms, with diminishing returns and length normalisation	17.3
Boosting	Building an ensemble sequentially, each weak model correcting the current errors	7.4
Bootstrap confidence interval	Interval from resampling the data with replacement many times and taking percentiles of the statistic	2.11
Bootstrapping (RL)	Updating an estimate using other estimates (e.g. V(s')) instead of waiting for the full return	14.6
Bootstrap sample	n rows drawn with replacement from n rows; about 36.8% of rows are left out	7.3
BPE (byte-pair encoding)	Subword tokenizer that repeatedly merges the most frequent adjacent symbol pair	12.8
Brier score	Mean squared error of predicted probabilities; a proper scoring rule for calibration	6.5
Broadcasting	NumPy rule for combining arrays of different shapes by stretching size-1 dimensions	3.2
Budget interference	A/B arms competing for the same campaign budgets and distorting each other's results	18.1

Term	Meaning	Where
Budget pacing	Spreading an ad budget over the day by throttling or boosting bids	14.11
C2PA	Industry standard for attaching tamper-evident provenance ('Content Credentials') to media	19.7
Cache	Fast store of recently computed results so repeated requests skip the work	16.4
Cache invalidation	Removing or refreshing cached entries when the underlying data changes	17.3
Calibration	Predicted probabilities matching observed frequencies; separate from ranking quality	6.5
Calibration within groups	A score of s means an s chance of the outcome in every group	19.2
Canary release	Giving a new model to a small slice of real traffic before ramping up	16.6
Candidate generation	First funnel stage that retrieves a few hundred plausible items from millions, judged by recall	13.8
Cannibalisation	A new feature stealing outcomes that would have happened anyway elsewhere on the site	18.9
Cardinality	Number of distinct values a categorical feature has	4.2
CART	Classification and Regression Trees: the binary-split tree algorithm used by scikit-learn	7.1
Cascade	Cheap checks on everything, expensive checks only on what earlier stages flag	18.3
Catalogue coverage	Share of all items that ever get recommended	13.11
CatBoost	Gradient-boosting library with ordered boosting and strong native categorical handling	7.4
Categorical cross-entropy	−ln of the probability the softmax gives the true class	10.3
Causal mask	Hides future positions so each token attends only to earlier ones (decoders)	12.7
CCPA / CPRA	California privacy laws giving rights to know, delete, correct and opt out	19.9
CDF (cumulative distribution function)	P(X ≤ x): share of values at or below x	10.2
Cell state (LSTM)	LSTM's long-term memory, updated additively so gradients survive many steps	12.4
Censored label	Outcome never observed because your own action (e.g. blocking a payment) prevented it	18.2
Central limit theorem (CLT)	Sample means are approximately normal around the true mean with spread σ/√n	2.10
Centroid	Mean point of a cluster	9.1
Chain rule	Derivative of nested functions = product of the local derivatives	2.4
Channel	One 2-D slice of an image or feature map (e.g. red, green, blue)	11.1
Chargeback	Card payment reversed after the cardholder disputes it, at the merchant's cost	4.1
Churn	Customers leaving or cancelling; a common prediction target	4.1
CI/CD	Continuous integration (automatic tests on every change) and continuous delivery (automated safe release)	16.10
Classification	Supervised task predicting a category	1.5
Class imbalance	One class much rarer than the other(s), as in fraud or disease	4.9
Class weights	Loss weights that make mistakes on the rare class cost more	4.9
CLIP	Image–text model trained contrastively so images and captions share one embedding space	18.6
[CLS] token	Special token whose final vector summarises the whole input for classification (BERT, ViT)	11.4
Clustering	Grouping similar examples together without labels	9.1
CNN (convolutional neural network)	Network built from convolution and pooling layers; the classic architecture for images	11.1
Cohen's kappa	Chance-corrected agreement between two annotators, (p_o − p_e)/(1 − p_e)	4.1
Cohort	Group of users sharing a trait (sign-up month, country); metrics per cohort reveal hidden failures	6.2
Cold start	Recommending with little or no history for a new user, item or system	13.10
Collaborative filtering (CF)	Recommending from other users' behaviour: people who liked what you liked also liked X	13.4
ColumnTransformer	scikit-learn tool applying different preprocessing to different column groups	3.5
COMPAS	US recidivism-risk score whose 2016 analysis sparked the fairness-impossibility debate	19.10
Compound scaling	Growing a network's depth, width and input resolution together in fixed ratios	11.4
Computational graph	Record of the operations that produced a value, used to compute gradients	10.4
Concept drift	Change in the relationship between inputs and the label, P(Y|X)	16.8
Conditional probability	Probability of A once you know B happened, P(A | B)	2.7
Condition number	Ratio of a matrix's largest to smallest stretch; large values make gradient descent slow and solutions unstable	5.4
Confidence (association rules)	Share of baskets with A that also contain B	9.8
Confidence interval (CI)	Range built by a procedure that captures the true value a stated share of the time	2.11

Term	Meaning	Where
Confounder	Hidden factor that drives two variables and makes them look causally linked	2.9
Confusion matrix	Table of TP, FP, FN and TN counts from which classification metrics are computed	6.3
Container	Packaged app plus its dependencies that runs the same anywhere (e.g. Docker)	16.10
Content-based filtering	Recommending items whose attributes resemble what the user liked	13.4
Contextual bandit	Bandit whose choice depends on context features (e.g. LinUCB)	14.4
Contextual embedding	Word vector that depends on the surrounding sentence (from models like BERT)	12.9
Context vector	Fixed-size summary of the input passed from encoder to decoder	12.6
Context window	Maximum number of tokens a model can read at once	12.8
Continuous training	Automatically retraining and validating models on new data	16.10
Contrastive loss	Loss pulling matching pairs together and pushing non-matching pairs apart (e.g. InfoNCE)	18.6
Conversion	The action an advertiser actually wants, such as a purchase or sign-up	18.1
Convex function	Bowl-shaped function with a single minimum, so gradient descent cannot get stuck in a worse one	2.5
Convolution	Sliding a small shared kernel over an input and summing element-wise products into a feature map	11.1
Co-occurrence	Items appearing together in the same sessions or baskets	17.1
Copy-on-write	pandas 3 behaviour where derived objects behave as copies, so edits never leak back silently	3.3
Core / border / noise point	DBSCAN roles: dense centre point / near a core point / isolated outlier labelled −1	9.3
Corpus	Collection of text used for training or analysis	12.8
Correlation	Strength of co-movement; Pearson for linear, Spearman for monotonic (ranks)	2.9
Cosine similarity	Dot product divided by both lengths; compares direction only	2.1
Cost-complexity pruning	Removing tree branches that don't pay for their leaves, controlled by ccp_alpha	7.1
Counterfactual explanation	Smallest actionable input change that flips a decision	19.5
Covariance matrix	Matrix of how each pair of features varies together; PCA takes its eigenvectors	9.5
CPC (cost per click)	Pricing where the advertiser pays only when the ad is clicked	18.1
CPU (central processing unit)	A computer's general-purpose processor; fine for classic ML and small models	3.7
Credit assignment	Working out which earlier actions deserve credit for a delayed reward	14.1
C (regularisation parameter)	Inverse regularisation strength: large C fits training data tightly, small C prefers a simpler boundary	8.2
Cross-attention	Attention with queries from the decoder and keys/values from the encoder	12.7
Cross-correlation	What deep-learning 'convolution' actually computes: sliding the kernel without flipping it	11.1
Cross-encoder	Model reading query and item together for an accurate but expensive relevance score	17.3
Cross-entropy	Average surprise $-\Sigma\ p \log q$ when reality is p and the prediction is q; the classification loss	2.12
Cross-fitting	Computing a feature (e.g. target encoding) for each fold from the other folds only, to avoid leakage	4.4
Cross-modal retrieval	Searching one kind of data (video) with another (text)	17.4
Cross-validation (CV)	Rotating the validation role across k folds and averaging the scores	6.6
CSAM	Child sexual abuse material; handled by a dedicated, legally governed process	18.3
CSAT (customer satisfaction)	Survey score of how satisfied customers are	15.1
CSV	Comma-separated values: a plain-text table format	3.3
CTR (click-through rate)	Clicks divided by impressions	18.1
CUPED	Variance-reduction method that adjusts experiment metrics with pre-experiment data	16.9
Curse of dimensionality	In high dimensions space empties and distances concentrate, so "nearest" loses meaning	8.1
Cyclical encoding	Encoding periodic values as (sin, cos) so December sits next to January	4.6
DAG (directed acyclic graph)	Steps connected by one-way dependencies with no cycles; how pipelines are described	16.10
Data augmentation	Creating extra training examples with label-preserving transformations	10.7
Data drift	Change in the input distribution P(X) (covariate shift)	16.8
DataFrame / Series	pandas table with labelled columns / a single labelled column	3.3
Data leakage	Information unavailable at prediction time reaching training or evaluation	4.8
Data minimisation	Collecting and keeping only the personal data you actually need	19.9
Data poisoning	Attack that corrupts training data, possibly planting a backdoor trigger	19.7

Term	Meaning	Where
Dataset / DataLoader	PyTorch classes that give single examples / shuffle and batch them for training	10.1
Datasheet for datasets	Document describing how a dataset was created, what it contains and how it should be used	19.8
Data validation (TFDV, Great Expectations)	Automatic checks that data matches the expected schema and ranges before it is used	16.10
Data versioning (DVC)	Tracking dataset versions like code so any model can be reproduced	16.10
Data warehouse / data lake	Store of cleaned tables for analysis / store of raw files of any kind	16.2
DAU / MAU	Daily / monthly active users	17.1
Davies–Bouldin / Calinski–Harabasz	Internal clustering scores: cluster overlap (lower is better) / between- vs within-cluster spread (higher is better)	9.9
DBSCAN	Density-based clustering: core points with ≥ min_samples neighbours within eps; others border or noise	9.3
DCG (discounted cumulative gain)	Sum of relevance scores, each divided by $\log_2$(position + 1), so lower positions count less	6.4
Deadly triad	Function approximation + bootstrapping + off-policy learning, which can make RL diverge	14.8
Decision boundary	Where the classifier's decision changes; for logistic regression, $w \cdot x + b = 0$	5.8
Decision stump	Tree with a single split; the classic weak learner in boosting	7.4
Decision tree	Model predicting by a sequence of feature-threshold questions ending in leaves	7.1
Deepfake	Fake video, audio or image impersonating a real person	19.7
Deep learning	Machine learning with many-layer neural networks that learn their own features	1.1
Degenerate feedback loop	Model's own outputs narrowing its future data until it only sees what it already likes	16.8
Delayed feedback	Labels (conversions, chargebacks) arriving long after the prediction	18.1
Demographic parity	Fairness criterion requiring equal approval (selection) rates across groups	19.2
Dendrogram	Tree of agglomerative merges; slicing it at a chosen height yields the clusters	9.2
Dense (fully connected) layer	Layer where every output is connected to every input	10.1
Depthwise-separable convolution	Per-channel spatial convolution followed by a 1×1 convolution; far cheaper than a full convolution	11.1
Derivative	Rate at which a function's output changes as its input changes; the slope	2.4
Determinant	Single number saying how much a square matrix scales area or volume; 0 means it has no inverse	2.2
DETR	Transformer object detector that predicts a set of boxes directly, without anchors or NMS	11.7
Dice loss	Segmentation loss based on mask overlap, robust to small foreground areas	11.7
Differencing	Replacing a series by its step-to-step changes to remove trend	12.10
Differential privacy	Guarantee that an algorithm's output barely changes whether any one person is in the data (parameter ε)	19.6
Diffusion model	Generative model that learns to turn random noise into an image step by step	11.7
Dimension	Number of entries in a vector, or one axis of an array	2.1
Dimensionality reduction	Describing data with fewer numbers (PCA, t-SNE, UMAP)	9.5
Discount factor (γ)	Weight on future rewards; horizon ≈ $1/(1-\gamma)$	14.2
Disparate impact ratio	Selection rate of one group divided by another's; below 0.8 fails the four-fifths rule of thumb	19.2
Disparate treatment / impact	Treating people differently because of a protected attribute / a neutral rule with unequal effect	19.4
Distillation	Training a small student model to mimic a large teacher	16.6
Distributional hypothesis	Words that appear in similar contexts tend to have similar meanings	12.9
Distribution shift	Production data differing from training data	16.2
Domain randomisation	Randomising a simulator's details so a policy transfers to the real world	14.11
Dot product	Sum of element-wise products of two vectors; large when they point the same way	2.1
Double DQN / Dueling DQN	DQN variants that reduce over-estimated values / separate state value from action advantage	14.8
DPIA (data protection impact assessment)	GDPR risk assessment required before high-risk processing	19.9
DPO	Direct Preference Optimization: trains a policy on preference pairs without a separate reward model or RL loop	14.10
DP-SGD	Training with per-example gradient clipping plus Gaussian noise to get differential privacy	19.6
DQN	Deep Q-Network: neural Q-function trained with experience replay and a target network	14.8
Dropout	Randomly zeroing units during training so the network can't rely on co-adapted features	10.7
dtype	Data type of an array's elements (e.g. int64, float32); sets memory use and precision	3.2

Term	Meaning	Where
Dual form (SVM)	Rewriting the SVM problem so it uses only dot products between examples, which enables kernels	8.2
Dummy-variable trap	One-hot columns summing to 1 are collinear with the intercept; drop one for unregularised OLS	4.4
Dwell time	How long a user stays on an item or page	13.1
Dynamic programming (RL)	Computing optimal values from a known model by repeated Bellman backups	14.5
Early stopping	Halting training when validation loss stops improving for a set patience	6.10
ECOA / Regulation B	US credit law requiring lenders to give specific reasons for denials	19.9
EDA (exploratory data analysis)	First look at a dataset: shapes, types, missing values, distributions and relationships	4.2
Effect size	How big a difference is, separate from whether it is statistically significant	16.8
ε-greedy	Explore a random action with probability ε, otherwise exploit the best known	14.4
Eigenvector	Direction a matrix only stretches, $Av = \lambda v$; λ is the eigenvalue	2.3
ElasticNet	Linear regression with a mix of L1 and L2 penalties	5.7
Elbow method	Choosing k where adding clusters stops reducing inertia much	9.1
ELU / PReLU / SELU	ReLU variants with a smooth or learned slope for negative inputs	10.2
EM algorithm (expectation–maximisation)	Alternates estimating soft assignments (E-step) and refitting parameters (M-step); fits GMMs	9.4
Embedding	Short dense learned vector representing a discrete thing (word, user, item)	13.6
Encoder / decoder	Part of a network that compresses input into a representation / part that expands it into the output	11.7
Encoder-only / decoder-only model	Transformer that reads text for understanding (BERT) / that generates text left to right (GPT)	12.7
Ensemble	Combination of several models' predictions	7.5
Entropy	Average uncertainty of a distribution, $-\Sigma\ p \log_2 p$ (bits)	2.12
Episode	One run from a start state to an end state	14.1
Episodic vs continuing task	Task with a natural end / task that goes on forever	14.2
Epoch	One full pass over the training data	2.5
Equalized odds	Fairness criterion requiring equal TPR and FPR across groups	19.2
Equal opportunity	Fairness criterion requiring equal true-positive rates across groups	19.2
ESMM	Ads model predicting pCTR × pCVR on all impressions so the conversion head isn't trained only on clicked ads	18.1
Estimator	Any scikit-learn object with fit(); models and transformers are estimators	3.5
ETA (estimated time of arrival)	Predicted time until something arrives, e.g. a delivery	4.6
EU AI Act	EU regulation sorting AI systems into risk tiers; credit scoring and hiring are high-risk	19.9
Euclidean / Manhattan distance	Straight-line distance / sum of absolute coordinate differences	8.1
Evasion attack	Crafting inputs at prediction time so the model misclassifies them	19.7
Exact / perceptual hash	Fingerprint that changes with any edit / one that stays similar after small edits like crops	18.3
Expectation (expected value)	Probability-weighted average of a random variable, E[X]	2.6
Expected calibration error (ECE)	Bucket-size-weighted average gap between mean predicted probability and observed rate	18.1
Expected cost	Probability-weighted cost of a decision; used to set thresholds	18.2
Experience replay	Storing transitions and training on random mini-batches of them to decorrelate updates	14.8
Experiment tracking	Logging each run's code version, parameters, data and metrics (e.g. MLflow)	16.10
Explainability	Ability to say why a model made a prediction in human-checkable terms	19.5
Explained variance ratio	Share of total variance captured by each principal component	9.5
Explicit / implicit feedback	Direct ratings / behaviour such as clicks, plays and purchases	13.2
Exploding gradient	Gradients growing exponentially through layers or time steps, causing huge updates or NaN	10.6
Exploration–exploitation	Trade-off between trying options to learn and using the best-known option	14.4
Exponential smoothing (ETS)	Forecasting with weighted averages where recent values count more	12.10
Exposure bias	Logs only show outcomes for items the system chose to show	4.1
F1 score	Harmonic mean of precision and recall	6.3
Factorisation machine	Model scoring every pair of features through dot products of small learned vectors	18.1
Fail closed	On failure, block the action (don't publish, don't suggest) rather than proceed; used where errors harm people	18.7
Fail open	On failure, let the action proceed (approve, publish) rather than block it	16.2

Term	Meaning	Where
Fairlearn / AIF360	Open-source toolkits for measuring and mitigating unfairness (Microsoft / IBM origins)	19.3
Fairness audit	Computing metrics per group and comparing them	19.3
Fairness criterion	Precise statistical condition on predictions across groups	19.2
Fairness penalty	Loss term that grows with the gap between groups	19.4
Fairness through unawareness	Dropping the sensitive column; weak because proxies carry the same information	19.3
FAISS	Meta's library for fast exact and approximate vector search	8.1
Fallback	Pre-agreed cheaper answer used when the model fails (previous model, cached score, rule)	16.2
False negative / false positive	A real positive the model missed / a negative the model wrongly flagged	4.1
False-removal / appeal-overturn rate	Share of removals that were wrong (by audit) / share of appeals that win	18.3
Fan-out on write / read	Pushing a new post into followers' feeds at write time / pulling posts when a feed is read	17.2
Fβ score	Weighted harmonic mean of precision and recall; β > 1 favours recall	6.3
FCN (fully convolutional network)	Network with only convolutional layers, so it outputs a map (e.g. per-pixel labels)	11.7
Feature	Measurable input property used by a model	1.6
Feature cross	New feature combining two others (device × ad category)	13.8
Feature engineering	Creating new inputs that make the pattern easier to learn	4.6
Feature hashing	Mapping categories to a fixed number of buckets with a hash function (hashing trick)	4.4
Feature importance	Score of how much each feature contributes (impurity-based, permutation, SHAP)	7.3
Feature map	Output grid of one convolution kernel applied across the input	11.1
Feature store	System serving consistent features: offline history for training, online latest values for serving	16.4
FedAvg	Federated averaging: the server averages client model weights, weighted by client data size	19.6
Federated learning (FL)	Training across devices or institutions by averaging locally trained model updates (FedAvg)	19.6
Feedback loop	A model's outputs shaping its own future training data	16.8
Feed-forward network	Network where data flows one way from input to output with no loops	10.1
FGSM	Fast Gradient Sign Method: $x + \varepsilon \cdot \text{sign}(\nabla_x \text{ loss})$ creates an adversarial example	19.7
Filtered ANN	Nearest-neighbour search restricted by filters such as category or price	18.6
Filter (kernel)	Small learned weight grid slid over the input in a convolution	11.1
Fine-tuning	Continuing training of a pretrained model on your task with a small learning rate	11.5
Flatten	Reshaping a feature map into one long vector before dense layers	11.1
FLOPs	Floating-point operations: a count of arithmetic work, used to compare model compute cost	11.4
Focal loss	Detection loss that down-weights easy examples so training focuses on hard ones	18.7
Fold	One of the k parts data is split into for cross-validation	6.6
Forecast horizon	How far ahead a forecast predicts	12.10
Forget gate	LSTM gate deciding which cell-state entries to keep or erase	12.4
Forward pass	Running inputs through the network to get predictions and the loss	10.3
Foundation model	Large model pretrained on broad data and adapted to many tasks	11.8
Four-fifths rule	US rule of thumb flagging a selection-rate ratio below 0.8 between groups	19.2
FP-Growth	Frequent-itemset mining without candidate generation, using a prefix tree	9.8
Fraud ring	Group of colluding accounts, often visible as a dense cluster in a graph	18.2
Freezing layers	Keeping pretrained weights fixed while training only the new layers	11.5
Friends-of-friends (FoF)	Candidates two hops away in a social graph; closing triangles	18.4
FTRL	Follow-the-regularised-leader: online learning rule for huge sparse models such as ad CTR	18.1
Function approximation	Representing values or policies with a model (e.g. a neural net) instead of a table	14.8
Funnel	Multi-stage pipeline: cheap high-recall retrieval, heavy ranking, rule-based re-ranking	13.8
GAE (generalised advantage estimation)	Smoothed advantage estimate trading bias for variance with a parameter λ	14.6
GAM (generalised additive model)	Model that adds up one smooth function per feature; flexible yet readable	19.5
Gate	Sigmoid-controlled switch deciding how much information to let through	12.4
Gaussian process	Probabilistic model giving a prediction and an uncertainty for any input; a classic surrogate in Bayesian optimisation	6.9

Term	Meaning	Where
GBDT (gradient-boosted decision trees)	Gradient boosting with decision trees: shallow trees added one by one, each correcting the current errors	7.4
GDPR	EU General Data Protection Regulation: rules for processing personal data of people in the EU	19.9
GELU	Smooth ReLU-like activation $z \cdot \Phi(z)$ used in transformers	10.2
Generalisation	Performing well on new, unseen data	1.3
General-purpose AI model	EU AI Act term for broadly capable models such as LLMs, with their own documentation duties	19.9
Generative AI	Models that produce new content such as text, images, audio or code	1.1
Geohash / S2 cell	Codes naming cells of a hierarchical grid over the map, for fast location lookups	18.8
Gini coefficient	Inequality measure from 0 (perfectly even) to 1 (one item gets everything)	18.5
Gini impurity	$1 - \Sigma\, p_k^2$, the chance of mislabelling a random sample from the node's class mix	7.1
Global average pooling (GAP)	Averaging each feature map to one number before the classifier head	11.2
Global vs local explanation	Describes the model's overall behaviour / explains one prediction	19.5
GloVe / fastText	Word embeddings from global co-occurrence counts / from character n-grams (handles unseen words)	12.9
GMM (Gaussian mixture model)	Clustering as a mix of Gaussians fitted with EM, giving soft assignments	9.4
Gold questions	Items with known answers mixed into annotation work to check labeller quality	4.1
GPU (graphics processing unit)	Chip with thousands of small cores; does the matrix maths of deep learning far faster than a CPU	3.7
Gradient	Vector of partial derivatives; points in the direction of steepest increase	2.4
Gradient accumulation	Summing gradients over several small batches before one update, to mimic a bigger batch	10.8
Gradient boosting	Boosting where each new tree fits the negative gradient of the loss (residuals for MSE)	7.4
Gradient clipping	Rescaling gradients whose norm exceeds a threshold to prevent explosions	10.6
Gradient descent (GD)	Minimising a loss by repeated steps $w \leftarrow w - \eta \nabla L$	2.5
Graph-cluster randomisation	Assigning whole communities to one experiment arm to limit network interference	18.4
Graph embedding	Vectors for nodes of a graph learned so connected nodes are close (e.g. node2vec, PinSage)	13.6
Graph neural network (GNN)	Neural network over a graph of entities, used e.g. to spot fraud rings	18.2
Greedy algorithm	Takes the best-looking step now without looking ahead, as tree splitting does	7.1
Greedy decoding / sampling	Generating by always taking the most likely token / by drawing tokens from the predicted distribution	12.5
Grid search	Trying every combination of listed hyperparameter values with cross-validation	6.9
Gridworld	Toy RL environment of cells an agent moves between	14.1
Ground truth	The correct answer a prediction is judged against	11.7
GroupNorm	Normalisation over groups of channels in each example; works with tiny batches	10.7
Group-specific thresholds	Different decision thresholds per group, chosen to equalise a rate	19.4
Group split	Keeping all rows of one entity in the same split	4.7
GRPO	Group Relative Policy Optimization: compares several sampled answers per prompt against their group mean	14.10
GRU	Gated recurrent unit with update and reset gates; lighter than an LSTM	12.4
Guardrail metric	Metric that must not get worse in an experiment, even if the primary improves	16.9
Hallucination	Generated content that is false or unsupported	15.2
Hard-case mining	Collecting the examples the model gets wrong or is unsure about for labelling	18.7
Hard negative	Non-matching example that looks very like a match; sharpens what the model learns	4.1
Hard / soft voting	Combining classifiers by majority label / by averaging their probabilities	7.5
Hard- vs soft-margin SVM	No points allowed inside the margin / some violations allowed at a cost set by C	8.2
Hash function	Function mapping any input to a fixed-size number; the same input always gives the same output	16.9
HDBSCAN	Hierarchical DBSCAN: finds clusters of varying density without a single eps	9.3
Head (network)	Final task-specific layers on top of a shared network body (backbone)	11.2
Health check	Regular probe confirming a service is up and answering correctly	16.6
He initialisation	Weight variance 2/fan_in, designed for ReLU layers	10.6
Hessian	Matrix of second derivatives; describes the curvature of a loss surface	5.4
Heuristic	Simple rule of thumb that is usually good enough	1.2
Hidden layer	Layer between input and output whose values are not observed directly	10.1

Term	Meaning	Where
Hidden state	An RNN's memory vector, updated at each step	12.2
Hierarchical clustering	Clustering that builds a nested tree of merges (see agglomerative)	9.2
Hinge loss	max(0, 1 − y·f(x)); the SVM loss	8.2
HIPAA	US law protecting health information	19.9
Historical / representation / measurement bias	Bias from an unequal past / from under-sampled groups / from features measured unevenly across groups	19.1
Hit rate@k (Hit@K)	Share of cases where at least one relevant item is in the top k	13.11
HNSW	Hierarchical Navigable Small World graph index for approximate nearest-neighbour search	13.9
HOG (histogram of oriented gradients)	Hand-crafted image feature counting edge directions in small cells	8.2
Holdback	Small slice kept on the old experience for months to measure long-term effects	16.9
Homogeneity / differencing attack	Learning a secret because everyone in a group shares it / by subtracting two aggregate answers	19.6
Homoscedasticity	Errors having the same spread at every prediction level (a linear-regression assumption)	5.5
HTTP	The request–response protocol web services speak; 200 means success	15.1
Huber loss	Quadratic for small errors, linear for large; robust regression and DQN	10.3
Human oversight	People able to monitor, review and override an AI system's decisions	19.9
Hybrid recommender	Combining collaborative and content-based signals	13.4
Hybrid retrieval	Combining lexical and semantic retrieval results	17.3
Hydration (render-time)	Filling in the latest live data (stock, sold status, capacity) just before display	18.6
Hyperparameter	Setting chosen before training, e.g. learning rate, depth or C	1.6
Hyperplane	Flat boundary one dimension lower than the space (a line in 2-D, a plane in 3-D)	5.1
Hypothesis test	Checking whether an observed effect could plausibly be noise under a null hypothesis	2.11
ICE (individual conditional expectation)	One curve per row showing how its prediction changes as a feature varies	19.5
Idempotent	Safe to repeat: doing it twice has the same effect as once	18.4
Identity matrix	Square matrix with 1s on the diagonal and 0s elsewhere; multiplying by it changes nothing	2.2
i.i.d. (independent and identically distributed)	Examples drawn independently from the same distribution; the assumption behind random splits	6.6
ImageNet	Large labelled image dataset (1,000 classes) used to pretrain and benchmark vision models	11.4
Impossibility theorem (fairness)	With different base rates, calibration and equal error rates can't all hold unless the model is perfect	19.2
Impression	One showing of an item or ad on a user's screen	17.1
Impression discounting	Lowering scores of items shown many times without any action	18.4
Impurity	How mixed the classes in a tree node are (Gini, entropy) or how spread the targets are (variance)	7.1
Impurity-based importance (MDI)	Tree feature importance from total impurity decrease; biased toward high-cardinality features	7.3
Imputation	Filling missing values (mean, median, KNN, iterative)	4.3
In-batch negatives	Using the other items in a training batch as negatives for each user	13.7
Inception (GoogLeNet)	CNN with parallel branches of different filter sizes and 1×1 bottlenecks	11.4
Independence / separation / sufficiency	Fairness families: decision independent of group / errors independent of group given the truth / scores mean the same per group	19.2
Independence (statistical)	Two events are independent when knowing one tells you nothing about the other	2.7
Indexing lag	Delay between content arriving and becoming searchable	17.4
Individual / counterfactual fairness	Similar people get similar predictions / the prediction wouldn't change if only the person's group changed	19.2
Inertia (WCSS)	Sum of squared distances from points to their cluster centroid; lower is tighter	9.1
Inference	Using a trained model to make predictions	1.3
Inference path	Everything that runs to turn a request into a prediction	16.1
InfoNCE	Contrastive loss: a softmax over similarities that must pick the true partner out of the batch	18.6
Information gain	Entropy of the parent minus the weighted entropy of the children after a split	7.1
Instance-based learning	Predicting from stored similar examples rather than a fitted formula (e.g. k-NN)	1.8
Integrity layer	Rules and classifiers that demote or remove harmful content inside ranking	17.2

Term	Meaning	Where
Inter-annotator (inter-rater) agreement (IAA)	How often independent labellers agree; measured with e.g. Cohen's kappa	4.1
Interference (experiments)	Treated units affecting control units, which biases an A/B test	16.9
Interleaving	Merging two rankers' results into one list and seeing whose items get clicked	16.9
Interpretability	Synonym for explainability: how directly a person can understand how a model works	19.5
Intersectional group	Group defined by several attributes at once (e.g. older women)	19.2
Intra-list diversity	How different the items within one recommendation list are	13.11
Intrinsically interpretable model	Model readable directly, such as a linear model, small tree or GAM	19.5
Invalid traffic	Clicks and impressions from bots or click farms, removed before billing and training	18.1
Inverse propensity scoring / weighting (IPS / IPW)	Weighting each logged example by 1/P(it was shown/examined) to remove exposure, selection or position bias	17.3
Inverted dropout	Scaling surviving units by 1/(1 − p) during training so nothing changes at evaluation	10.7
Inverted index	Map from each word to the documents containing it; the core of keyword search	17.3
IoU (intersection over union)	Overlap of two boxes divided by their union; matches detections to ground truth	11.7
IQR (interquartile range)	Distance between the 25th and 75th percentiles; a spread measure that ignores extremes	2.8
Irreducible noise	Error no model can remove because the outcome is partly random	6.7
Isolation Forest	Anomaly detector: anomalies are isolated by random splits in fewer steps	9.7
Isotonic regression	Non-decreasing step-function calibration mapping; needs more data than Platt	6.5
Item2vec	word2vec's skip-gram applied to sessions or playlists to learn item embeddings	13.6
Iterative imputation (MICE)	Filling each missing column by predicting it from the others, repeated for several rounds	4.3
IVF (inverted file index)	ANN index that clusters vectors with k-means and searches only the nearest clusters (nprobe)	13.9
Jaccard similarity	$\|A \cap B\| / \|A \cup B\|$ for sets	13.3
Jupyter notebook	Interactive document mixing code cells, outputs and notes; good for exploration	3.1
k-anonymity	Every quasi-identifier combination shared by at least k records	19.6
KD-tree	Tree that splits space along coordinates to speed up nearest-neighbour search in low dimensions	8.1
Kernel PCA	PCA in an implicit non-linear feature space via the kernel trick	9.5
Kernel trick	Replacing dot products with a kernel $K(x, z) = \varphi(x) \cdot \varphi(z)$ without computing φ	8.2
Key-value (KV) store	Database that returns a stored value for a given key, very fast	18.4
k-fold cross-validation	Cross-validation with k equal folds, each used once for validation	6.6
Kill switch	Pre-built control that instantly turns a model or feature off	15.3
KL divergence	Extra surprise from predicting with q instead of p; non-negative and asymmetric	2.12
k-means	Clustering by alternately assigning points to the nearest centroid and recomputing centroids	9.1
k-means++	Initialisation picking centroids with probability $\propto$ squared distance to those already chosen	9.1
k-nearest neighbours (k-NN)	Predicting by majority vote or average of the k closest training points	8.1
KS test	Kolmogorov–Smirnov: maximum gap between two empirical CDFs, used for drift	16.8
KV cache	Stored keys and values of earlier tokens so a transformer doesn't recompute them when generating	12.7
L1 regularisation	Penalty $\lambda \Sigma \|w\|$ that drives some weights exactly to zero	5.7
L2 regularisation	Penalty $\lambda \Sigma w^2$ that shrinks all weights smoothly (weight decay)	5.7
Label	Target value to predict (y)	1.6
Label bias	Target that is a flawed or unevenly recorded stand-in for what you care about	19.1
Label encoding	Mapping categories to integer codes; `LabelEncoder` is meant for targets	4.4
Labelling function	Small rule or heuristic that votes a label; many are combined in weak supervision	4.1
Label shift	Change in the outcome distribution P(Y)	16.8
Lag feature	Past value of a series (yesterday, last week) used as an input	12.10
Lambda / kappa architecture	Separate batch and streaming paths / one streaming path for everything	16.4
LambdaMART	Gradient-boosted trees trained with a ranking objective that targets NDCG	17.3
Laplace mechanism	Adding Laplace noise with scale sensitivity/ε to achieve ε-differential privacy	19.6
Laplace smoothing	Adding α (usually 1) to every count so unseen words don't get zero probability	8.3
Lasso	Linear regression with an L1 penalty; performs feature selection	5.7

Term	Meaning	Where
Latency	Time from request to response; usually reported as percentiles	16.6
Latency budget	Milliseconds the whole prediction path may take, itemised by stage	16.6
Latent factors	Hidden dimensions learned by matrix factorisation that describe users' tastes and items' traits	13.5
Lawful basis	GDPR legal reason for processing personal data (consent, contract, legitimate interests ...)	19.9
LayerNorm	Normalisation across the features of each example; standard in transformers	10.7
Lazy learning	Doing no work at training time and all of it at prediction time (k-NN)	8.1
L-BFGS	Quasi-Newton optimiser using curvature estimates; scikit-learn's default logistic-regression solver	5.8
l-diversity / t-closeness	k-anonymity add-ons requiring varied sensitive values per group / a distribution close to the overall one	19.6
Leaf-wise tree growth	Splitting the leaf with the biggest gain next (LightGBM) rather than growing level by level	7.4
Leaky ReLU	ReLU variant with a small slope α for negative inputs	10.2
Learning curve	Training and validation score vs training-set size	6.8
Learning rate	Step size η of gradient descent	2.5
Learning-rate schedule	Changing the learning rate during training (step, cosine, warmup, one-cycle)	10.5
Learning to rank	Training models to order items (pointwise, pairwise, listwise losses)	17.3
LeCun initialisation	Weight variance 1/fan_in, suited to SELU and linear layers	10.6
Leetspeak / homoglyphs	Evasion by swapping letters for digits ('fr33') / for look-alike characters from other alphabets	18.3
LeNet / AlexNet / VGG	Early landmark CNNs (1998, 2012, 2014) that showed deep convolutional nets work	11.4
Lexical vs semantic retrieval	Matching exact words (BM25) / matching meaning via embeddings	17.3
Lift	Confidence(A → B) divided by support(B); above 1 means a positive association	9.8
LightGBM	Gradient-boosting library with leaf-wise histogram trees, GOSS and EFB	7.4
Light / heavy ranker	Cheap first-pass ranker over many candidates / expensive model over the survivors	17.2
Likelihood	How probable the observed data is under a given hypothesis or parameter value	2.7
LIME	Local explanation fitting a sparse linear surrogate around one prediction	19.5
Lineage	Record of which data, code and settings produced a model	16.2
Linear regression	Predicting a number as w·x + b by minimising squared error	5.2
Linkage (single / complete / Ward)	Rule for the distance between clusters: closest points / farthest points / increase in variance	9.2
Link prediction	Predicting which new edges will appear in a graph	18.4
LLM (large language model)	Very large transformer trained on huge amounts of text to predict the next token; powers chatbots and assistants	1.1
Load balancer	Component spreading incoming requests across many servers	15.1
Local differential privacy	Noise added on each user's device before data is sent	19.6
Local Outlier Factor (LOF)	Anomaly score comparing a point's local density with its neighbours'	9.7
Logistic regression (LR)	Linear score passed through a sigmoid to give a class probability	5.8
Logit	Raw pre-sigmoid or pre-softmax score; for logistic regression, the log-odds	5.8
Log loss	Negative log-likelihood of labels under predicted probabilities; binary cross-entropy	5.8
Log-odds	ln(p/(1 − p)); the quantity a logistic regression models linearly	5.8
logQ correction	Subtracting log(sampling probability) from logits so popular items aren't over-penalised as negatives	13.7
Loss function	Formula scoring how wrong a prediction is; training adjusts parameters to make it small (also called cost or objective)	1.6
LSTM	Recurrent cell with an additive cell state and forget, input and output gates	12.4
Macro / weighted average	Averaging per-class metrics equally / in proportion to class size	6.3
MAD (median absolute deviation)	Median distance from the median; an outlier-resistant spread measure	9.7
MAE	Mean absolute error; optimal constant prediction is the median	6.2
MAP	Mean average precision over queries or users	6.4
MAPE (mean absolute percentage error)	Average error as a percentage of the true value; breaks near zero	6.2
mAP (mean average precision)	Average precision averaged over classes (and IoU thresholds); the standard detection metric	11.7
Marginal recall	Extra relevant items a retrieval source finds that the other sources miss	17.1
Margin (SVM)	Gap between the decision boundary and the closest training points; SVMs maximise it	8.2

Term	Meaning	Where
Market-basket analysis	Finding items often bought together, via association rules	9.8
Markov property	The next state depends only on the current state and action, not the full history	14.2
matplotlib / seaborn	Python plotting libraries; seaborn adds statistical charts on top of matplotlib	3.1
Matrix	Rectangular grid of numbers; a dataset is a matrix with one row per example	2.2
Matrix factorisation (MF)	Approximating the user–item matrix by user and item factor vectors whose dot product predicts affinity	13.5
Matrix inverse	Matrix A^{-1} that undoes A, so $A^{-1}A = I$; in practice solve the system instead of computing it	2.2
Maximum likelihood estimation (MLE)	Choosing parameters that make the observed data most probable	5.2
Max pooling	Downsampling by keeping the largest value in each window	11.2
MCAR / MAR / MNAR	Missing completely at random / at random given other columns / not at random (depends on the missing value itself)	4.3
MDP (Markov decision process)	Formal RL setting: states, actions, transition probabilities, rewards and a discount factor	14.2
Mean / median / mode	Average value / middle value when sorted / most frequent value	2.8
Mean pooling	Averaging several vectors (e.g. frame embeddings) into one	17.4
Membership inference	Attack deciding whether a record was in the training set	19.7
Meta-model	Model trained on other models' predictions, as in stacking	7.5
Micro-batching	Grouping a few requests into one model call to use hardware efficiently	17.2
Mini-batch	Small subset of examples used for one gradient update	5.4
Minimum detectable effect (MDE)	Smallest lift an experiment is sized to detect; sample size $\propto 1/\text{MDE}^2$	16.9
Minkowski / Chebyshev / Hamming distance	General Lp distance / largest coordinate difference / count of positions that differ	8.1
Min-max scaling	Rescaling a feature to [0, 1] with (x − min)/(max − min)	4.5
Missing indicator	Extra 0/1 column marking where a value was missing, so the model can use missingness itself	4.3
Mixup / CutMix	Augmentations blending two images and their labels / pasting a patch of one image onto another	11.6
ML (machine learning)	Getting a computer to learn a task's rules from examples instead of having a person write them	1.1
ML objective	The measurable quantity the model predicts or optimises, chosen to serve the business goal	15.1
MLOps	DevOps practices (versioning, testing, CI/CD, monitoring) applied to data and models	16.10
MLP (multi-layer perceptron)	Feed-forward network of fully connected layers with non-linear activations	10.1
MMoE (multi-gate mixture of experts)	Multi-task architecture where each task head mixes shared expert sub-networks with its own gate	17.2
MMR (maximal marginal relevance)	Greedy re-ranking trading relevance against similarity to items already chosen	13.8
MNIST / CIFAR-10 / COCO	Classic benchmark datasets: handwritten digits / small colour images / everyday-object detection	11.3
MobileNet / EfficientNet	Efficient CNNs built from depthwise-separable convolutions / scaled with compound scaling	11.4
Model-based learning	Learning a compact model (e.g. a formula with weights) and throwing the training data away afterwards	1.8
Model card	Short document with a model's intended use, per-group evaluation and limits	19.8
Model extraction	Copying a model by querying its API and training a surrogate	19.7
Model-free RL	Learning values or policies from experience without knowing the transition probabilities	14.3
Model inversion	Attack reconstructing sensitive inputs or features from a model's outputs	19.7
Model registry	Store of model versions with metrics, lineage, stages and owners	16.10
Momentum	Optimizer that accumulates a velocity of past gradients	10.5
Monte Carlo (MC, RL)	Updating value estimates toward the actual return at the end of an episode	14.6
MRR (mean reciprocal rank)	Mean of 1/rank of the first relevant result	6.4
MSE	Mean squared error; optimal constant prediction is the mean	6.2
Multiclass vs multi-label	One label out of K classes / any number of labels per example	5.9
Multicollinearity	Features that are near-copies of each other, making coefficients unstable	5.5
Multi-head attention	Several attention heads in parallel subspaces, concatenated and projected	12.7
Multimodal	Using several kinds of input together (image, text, audio)	18.3
Multiple comparisons (p-hacking)	Running many tests makes some look significant by chance; correct with e.g. Bonferroni	2.11
Multi-task learning (MTL)	One shared network body with a head per task or event	17.2
Naive Bayes	Bayes' theorem with features assumed independent given the class	8.3
NaN	'Not a number': the floating-point marker for a missing or undefined value	3.3

Term	Meaning	Where
Natural / behavioural labels	Outcomes the world records anyway (delivery time) / labels from user behaviour (clicks)	16.3
NDCG	Normalised discounted cumulative gain: position-discounted graded relevance ÷ ideal	6.4
Negative downsampling	Keeping only a fraction of negatives to cut training cost; probabilities must then be corrected	4.9
Negative sampling	Training against a few sampled negatives instead of a full softmax	12.9
Nested cross-validation	Outer CV loop around the whole tuning procedure for an unbiased estimate	6.6
Nesterov momentum	Momentum that evaluates the gradient at the look-ahead position	10.5
Netflix Prize	2006–2009 competition on movie-rating prediction that popularised matrix factorisation	13.5
Neural architecture search (NAS)	Automatically searching for a good network design	11.4
Neural network	Model made of layers of simple units (neurons) whose weights are learned by gradient descent	10.1
Neuron (artificial)	Unit computing a weighted sum of inputs plus a bias, then applying an activation	10.1
NLP (natural language processing)	Field of ML working with human language	12.8
NMI (normalised mutual information)	Agreement between a clustering and known labels, from 0 (none) to 1 (perfect)	9.9
NMS (non-maximum suppression)	Keeping the most confident detection and removing boxes that overlap it above an IoU threshold	11.7
Node / branch / leaf	Point where a tree asks a question / one answer path / end point that gives the prediction	7.1
Non-parametric model	Model whose size grows with the data instead of being fixed in advance (k-NN, trees)	8.1
Non-stationarity	The environment or payoffs changing over time	14.4
Normal equation	Closed-form least-squares solution $\theta = (X^T X)^{-1} X^T y$	5.3
Normal (Gaussian) distribution	Symmetric bell-shaped distribution set by a mean μ and standard deviation σ	2.6
Normalised entropy	Log loss divided by the log loss of always predicting the average rate; lower is better	18.1
Norm (L1, L2, L∞)	Length of a vector: sum of absolute values (L1), straight-line length (L2) or largest entry (L∞)	2.1
Novelty effect	Temporary change in behaviour because something is new; fades over time	16.9
Novelty / serendipity	Recommending less-popular items / relevant items the user would not have found alone	13.11
n-step return / TD(λ)	Targets that look n steps ahead / blend all step counts with weight λ	14.6
Null hypothesis (H_0)	Default 'no effect' claim that a hypothesis test tries to reject	2.11
Numerical gradient check	Comparing a hand-coded gradient with (f(x+h) − f(x−h))/2h to catch bugs	2.4
NumPy	Python library for fast n-dimensional arrays and maths on them	3.2
NYC Local Law 144	New York City rule requiring annual bias audits of automated hiring tools	19.9
Object detection	Finding objects in an image as boxes with class labels and confidences	11.7
Object-level recall	Share of real objects caught in every view they appear in, not per box	18.7
Observation window	Waiting period before a missing event is recorded as a negative label	17.1
OCR (optical character recognition)	Reading text out of images or video frames	17.4
Odds	p/(1 − p): how much likelier an event is to happen than not	4.9
Odds ratio	e^w for a logistic-regression coefficient: factor by which the odds change per unit	5.8
Offline RL	Learning a policy only from logged data, without new interaction	14.12
Off-policy	Learning about one policy (e.g. greedy) while following another (e.g. ε-greedy)	14.7
OLS (ordinary least squares)	Linear regression fitted by minimising the sum of squared errors	5.3
One-hot encoding	One 0/1 column per category	4.4
One-vs-one	Training a binary classifier for every pair of classes and letting them vote	5.9
One-vs-rest	K binary classifiers, one per class against the rest	5.9
Online inference	Computing a prediction at request time	16.5
Online learning	Updating a model continuously from small chunks of data (`partial_fit`)	1.7
ONNX	Open format for exporting models so they run in other runtimes	16.6
On-policy	Learning about the same policy that is choosing the actions (e.g. SARSA)	14.7
OOV (out-of-vocabulary)	A word missing from the vocabulary; subword tokenisers avoid the problem	12.8
Open-weights model	Model whose trained weights are published so you can host it yourself	15.1
Operating point	The specific threshold, and its precision/recall, a deployed model actually uses	6.3
Optimizer	Algorithm that updates the parameters from the gradients (SGD, Adam, ...)	5.1
Optuna	Python library for hyperparameter tuning with Bayesian-style search and early pruning	6.9

Term	Meaning	Where
Orchestration	Scheduling and running pipeline steps in order (Airflow, Kubeflow, Dagster)	16.10
Ordered boosting	CatBoost trick computing each row's statistics only from earlier rows to avoid target leakage	7.4
Ordinal encoding	Mapping ordered categories to integers (small < medium < large)	4.4
Outlier	Value far from the rest; may be an error or the most interesting case	4.3
Outlier vs novelty detection	Finding anomalies inside the training data / flagging new points unlike a clean training set	9.7
Out-of-bag (OOB) score	Validation from the rows each bootstrap tree never saw	7.3
Out-of-core learning	Training on data too big for memory by streaming it in chunks	1.7
Out-of-fold prediction	Prediction for a row made by a model that did not train on that row's fold	7.5
Overfitting	Fitting training noise; low training error, much higher validation error	1.10
Oversampling / undersampling	Repeating rare-class rows / dropping common-class rows to rebalance training data	4.9
p95 / p99 latency	Response time that 95% / 99% of requests beat; captures the slow tail	16.6
Pacing	Control loop that spreads a campaign's spend over its planned period	18.1
Padding (same / valid)	Adding border pixels so the output keeps the input size ('same') or not adding any ('valid')	11.1
pandas	Python library for tables (DataFrames): loading, cleaning, grouping, joining	3.1
Parameter (model)	Number the model learns during training, such as a weight or bias	1.3
Partial dependence (PDP)	Average prediction as one feature is varied over a grid	19.5
Partial derivative	Derivative with respect to one input while holding the others fixed	2.4
Partial observability	The agent sees only part of the true state	14.9
PCA	Projection onto orthogonal directions of maximum variance	9.5
pCTR / pCVR	Predicted click-through rate / predicted conversion rate given a click	18.1
Pearson / Spearman correlation	Linear correlation of values / correlation of their ranks (catches any monotonic trend)	4.2
Percentile	Value below which a given share of the data falls; the 90th percentile (p90) beats 90% of values	2.8
Perceptron	Single neuron with a step activation; can't learn XOR	10.1
Permutation importance	Drop in held-out score when one feature's values are shuffled	7.3
Perplexity (t-SNE)	t-SNE setting roughly equal to how many neighbours each point considers	9.6
PGD (projected gradient descent attack)	Repeated FGSM-style steps kept within the budget; the standard strong attack	19.7
PII (personally identifiable information)	Data that identifies a person directly or in combination (name, email, ID number)	16.3
Pipeline	scikit-learn object chaining preprocessing and a model into one estimator	3.5
Platt scaling	Calibration by fitting a logistic curve on the model's scores	6.5
PMF / PDF	Probability mass function (discrete values) / probability density function (continuous values)	2.6
Point-in-time join	Joining each training example to feature values as they were at its timestamp	4.8
Pointwise / pairwise / listwise	Learning-to-rank losses on single items / on item pairs / on whole lists	17.3
Policy	Rule mapping states to actions (deterministic or stochastic)	14.3
Policy evaluation / improvement / iteration	Computing a policy's values / making it greedy on those values / repeating both until stable	14.5
Policy gradient	Gradient ascent on expected return, raising the log-probability of good actions	14.9
Pooling	Parameter-free downsampling of feature maps (max or average)	11.2
Popularity bias	Recommenders over-recommending already-popular items	13.12
Population / sample / statistic	Everyone you care about / the part you measured / a number computed from the sample	2.10
Positional encoding (PE)	Position information added to token embeddings because attention ignores order	12.7
Position bias	Items shown higher get more clicks regardless of relevance	17.3
Posterior	Updated belief after combining the prior with the data via Bayes' theorem	2.7
Post-hoc explanation	Method that explains an already-trained black-box model from outside	19.5
PPO	Proximal Policy Optimization: actor-critic that clips how far each update moves the policy	14.9
PR-AUC	Area under the precision–recall curve; the metric for rare positives	6.3
Pre-activation	A neuron's weighted sum z = w·x + b before the activation function	10.1
Precision	TP/(TP + FP): share of flagged items that are truly positive	6.3
Precision@k	Share of the top k results that are relevant	6.4

Term	Meaning	Where
Precision–recall (PR) curve	Plot of precision against recall as the threshold varies; best view for rare positives	6.3
Prediction (ŷ)	The model's output for one input	1.6
Predictive parity	Equal precision (PPV) across groups	19.2
Pre-ranking	Light scoring step between retrieval and the heavy ranker	13.8
Prevalence (of violating views)	Share of all views that land on violating content, estimated by sampling	18.3
Principal component	New axis in PCA; a linear combination of the original features	9.5
Prior	Belief about something before seeing the current data	2.7
Prioritised replay	Sampling replay transitions with large TD errors more often	14.8
Privacy budget (composition)	Total ε spent: each differentially private query adds to it	19.6
Probability distribution	Rule giving how likely each value of a random variable is	2.6
Product quantisation (PQ)	Compressing vectors into short codes for memory-efficient approximate search	13.9
Prompt / prompt injection	Text instructions sent to an LLM / an attack hiding instructions in input the model reads	15.2
Proper scoring rule	Metric (log loss, Brier) that is best when predicted probabilities equal the true ones	6.5
Protected (sensitive) attribute	Characteristic such as race, sex or age that should not drive decisions	19.1
Provenance	Verifiable record of where data or content came from and how it changed	19.7
Proxy feature	Feature that stands in for a sensitive attribute (e.g. ZIP region for group)	19.3
Proxy label	Easier-to-get stand-in for the outcome you care about, e.g. clicks for satisfaction	4.1
Proxy trap	Optimising an easy metric (clicks) that drifts away from the real goal (satisfaction)	17.1
Pruning (trees)	Stopping growth early or cutting back branches to reduce overfitting	7.1
Pseudonymisation	Replacing identifiers (e.g. hashing IDs); still personal data, not anonymous	19.6
PSI (population stability index)	Σ(cur − ref)·ln(cur/ref) over bins; > 0.25 means investigate	16.8
p-value	Probability of data at least this extreme if the null hypothesis were true	2.11
PYMK (People You May Know)	Social-network feature suggesting new connections	18.4
PyTorch	Python deep-learning framework with tensors, autograd and GPU support	3.7
Q-learning	Off-policy TD control updating Q toward r + γ·max Q(s', ·)	14.7
QPS (queries per second)	Request rate a service must handle; plan for peak, not average	15.3
Quantile	Cut point splitting sorted data at a given fraction (the 0.9 quantile = 90th percentile)	4.6
Quantisation	Storing weights/activations with fewer bits (e.g. int8) to shrink and speed up a model	16.6
Quasi-identifier	Attribute harmless alone but identifying in combination (ZIP code, birth date, sex)	19.6
Query, key, value (Q, K, V)	Attention's three projections: what a position looks for, what it offers to match, what it contributes	12.6
Query understanding	Normalising, spell-correcting, expanding and classifying a search query before retrieval	17.3
R^2	Coefficient of determination: share of target variance explained, 1 − SSE/SST	6.2
RAG (retrieval-augmented generation)	Retrieving relevant documents and giving them to an LLM so its answer is grounded in them	12.9
RAM	A computer's working memory; data must fit in it for fast processing	5.3
Random forest (RF)	Bagged decision trees with a random subset of features considered at each split	7.3
Random search	Tuning by sampling hyperparameter combinations from distributions	6.9
Random variable	Quantity whose value comes from a random process, such as a die roll	2.6
Ranking	Ordering items for a user or query; also the precise funnel stage after retrieval	13.8
RBF kernel	Gaussian similarity $\exp(-\gamma\|x - z\|^2)$; γ sets how local each point's influence is	8.2
Reach-aware re-scoring	Re-checking content with a stronger model each time its views cross a tier	18.3
Real-time bidding	Auctioning each ad impression in milliseconds as a page loads	14.11
Reason codes	Plain-language top reasons given to a customer for an adverse decision	19.5
Recall	TP/(TP + FN): share of true positives that were caught (sensitivity)	6.3
Recall@k	Share of relevant items that appear in the top k	6.4
Receptive field	Region of the input that can influence a unit	11.2
Reciprocal rank fusion (RRF)	Merging ranked lists by summing 1/(k + rank) per list (k ≈ 60); needs no training or score calibration	17.4
Recommender system	System predicting which items a user will want from a large catalogue	13.1

Term	Meaning	Where
Reconstruction error	How far data is from its compressed-then-restored version	9.5
Recourse	Telling a person what they could change to get a different decision	19.5
Red-teaming	Deliberately attacking your own system to find failures before others do	19.7
Reformulation / abandonment	User rewriting the query / leaving without clicking; both signal dissatisfaction	17.3
Regression	Supervised task predicting a number	1.5
Regression suite	Fixed test set re-run on every new version to catch quality going backwards	18.6
Regret	Total reward lost compared with always choosing the best option	14.4
Regularisation	Anything that limits a model's freedom to fit noise (penalties, dropout, early stopping)	5.7
REINFORCE	Basic policy-gradient algorithm weighting log-probability gradients by the return	14.9
Reinforcement learning	Learning a policy by trial and error to maximise long-run reward	14.1
Relevance floor	Minimum relevance a result (or ad) must clear before it can be shown	17.3
Reliability diagram	Plot of mean predicted probability vs observed frequency per bin	6.5
ReLU	max(0, z); the default hidden-layer activation	10.2
Replica	Extra copy of a service or data store, for capacity and safety	16.6
Reporter reputation	Historical precision of a user's reports, used to weight new reports	18.3
Representation (metric) learning	Learning embeddings where distance reflects similarity	18.6
Re-ranking	Final funnel stage applying diversity, freshness and business rules to the ranked list	13.8
Residual	Difference between a true value and the prediction	5.5
Residual connection	Skip path y = x + F(x) that gives gradients an identity highway	11.4
ResNet	CNN family with residual (skip) connections, trainable at 100+ layers	11.4
Response bias	Survey answers skewed by who chooses to respond or how the question is asked	17.2
Responsibility (GMM)	Probability that a point belongs to each Gaussian component	9.4
Retrieval	Candidate generation: narrowing millions of items to hundreds, graded on recall	17.1
Return (RL)	Discounted sum of future rewards $G_t = r_{t+1} + \gamma G_{t+1}$	14.2
Reward hacking	Agent maximising the reward as written rather than the intended goal	14.12
Reward model (RM)	Model trained on human preference comparisons to score responses in RLHF	14.10
Reweighing	Weighting (group, label) cells so group and label are independent in the weighted data	19.4
Rich-get-richer	Feedback loop where already-popular items or people get ever more exposure	18.4
Ridge regression	Linear regression with an L2 penalty	5.7
Right to erasure	GDPR right to have your personal data deleted ('right to be forgotten')	19.9
RLAIF / Constitutional AI	RLHF where an AI model, guided by written principles, supplies the preference labels	14.10
RLHF	Reinforcement learning from human feedback: SFT, reward model, then PPO with a KL penalty	14.10
RL with verifiable rewards	RL where rewards come from automatic checks (tests pass, maths answer correct)	14.10
RMSE	Root mean squared error, in the target's units; weighs big misses more	6.2
RMSProp	Optimizer dividing each step by a running RMS of recent gradients	10.5
RNN (recurrent neural network)	Network that reads a sequence step by step, carrying a hidden state	12.2
Robust scaling	Rescaling with the median and IQR so outliers have little influence	4.5
ROC curve	Plot of true-positive rate against false-positive rate as the threshold varies	6.3
ROI (return on investment)	Value gained per unit of money spent	18.9
Rollback	Returning quickly to the previous model, features or config after a bad release	13.7
Rolling deploy	Replacing servers with the new version a few at a time	16.6
Row subsampling	Training each boosting tree on a random fraction of rows to reduce overfitting	7.4
RPM (revenue per mille)	Revenue per thousand impressions	18.1
Rules engine	System applying explicit human-written conditions, alongside or before a model	18.2
Sampled metrics	Ranking metrics computed against a few sampled negatives instead of the full catalogue; can mislead	13.11
Sample inefficiency	Needing very many interactions to learn	14.7
Sample ratio mismatch (SRM)	Arms receiving a different share of users than planned; signals a broken experiment	16.9
SAM (Segment Anything Model)	Meta's promptable segmentation foundation model	11.7

Term	Meaning	Where
SARIMA	ARIMA with extra seasonal terms	12.10
SARSA	On-policy TD control updating Q toward r + γ·Q(s', a') for the action actually taken	14.7
Scaled dot-product attention	softmax($QK^T/\sqrt{d_k}$)V	12.6
ScaNN / Annoy / hnswlib	Popular open-source ANN search libraries	13.9
Schema check	Validating that incoming data has the expected columns, types and ranges	16.8
scikit-learn	Python library of classic ML models, preprocessing and evaluation tools with a shared fit/predict API	3.5
Scree plot	Chart of explained variance per component, used to pick how many to keep	9.5
Seasonal naive	Forecast repeating the value from one season ago (e.g. same weekday last week)	12.10
Second-price / GSP / first-price auction	Winner pays the runner-up's bid / multi-slot version of that / winner pays its own bid	18.1
Secure aggregation	Protocol letting the server see only the sum of client updates, never one client's	19.6
Selection bias	Distortion caused by which examples ended up in the data	2.9
Selective labels	Outcomes seen only for people the system approved, so the rest stay unknown	19.1
Self-attention	Attention where queries, keys and values all come from the same sequence	12.7
Self-play	Training an agent by having it play against copies of itself	14.11
Self-supervised learning	Learning from labels generated from the data itself (e.g. next-token prediction)	1.4
Semantic / instance segmentation	Labelling every pixel with a class / separating each individual object's pixels	11.7
Semi-supervised learning	Learning from a few labelled examples plus many unlabelled ones	1.4
Sensitivity (DP)	Largest change one person's record can make to a query result	19.6
Sentence embedding	One vector representing a whole sentence or document	12.9
Seq2seq	Encoder–decoder model mapping an input sequence to an output sequence	12.5
Sequence data	Data whose order matters: text, audio, time series, event streams	12.1
Sequential recommendation	Predicting the next item from the order of a user's recent items	18.5
Sequential test	Experiment analysis that allows checking results early without inflating false positives	16.9
Session embeddings	Item vectors learned by skip-gram over browsing sessions, so items considered together are close	18.9
SFT (supervised fine-tuning)	Training a pretrained LLM on example prompts and good answers before RL	14.10
Shadow deployment	New model scores live traffic in parallel; outputs are logged but not used	16.6
SHAP	Shapley-value attributions splitting one prediction into additive feature contributions	19.5
Shapley values	Game-theory split of a prediction among features by average marginal contribution	19.5
Share velocity	How fast an item is being shared; a sign of fast-spreading content	18.3
Sigmoid	$\sigma(z) = 1/(1 + e^{-z})$, squashing a score into (0, 1)	2.4
Significance level (α)	False-positive rate you accept in a hypothesis test, often 0.05	2.11
Silhouette score	Average over points of (b − a)/max(a, b), where a = own-cluster distance and b = nearest-other-cluster distance	9.1
SiLU (Swish)	Smooth activation $z \cdot \sigma(z)$, common in modern vision models and LLMs	10.2
Sim-to-real gap	Performance lost when a policy trained in simulation meets reality	14.12
Singular value	Stretch factor on the diagonal of Σ in the SVD; large ones carry most of the matrix's information	2.3
Skip-gram / CBOW	word2vec variants: predict context words from the centre word / the centre word from its context	12.9
SLA (service-level agreement)	Promise to a customer about a service level, often with penalties	18.2
SLO (service-level objective)	Internal target for a service, such as p99 latency under 100 ms	15.1
SMOTE	Oversampling by creating synthetic minority points between minority neighbours	4.9
Snorkel	Weak-supervision library that combines labelling functions into training labels	4.1
Softmax	$e^{z_k}/\Sigma e^{z_j}$: turns K scores into probabilities that sum to 1	5.9
Sparsity (user–item matrix)	Most user–item cells are empty because each user touches few items	13.1
Specificity	TN/(TN + FP): share of real negatives correctly left alone	6.3
Spectrogram	Image of how a sound's frequencies change over time	18.5
Splines	Piecewise polynomials joined smoothly, used to fit curves flexibly	5.6
Stacking	Meta-model trained on out-of-fold predictions of diverse base models	7.5
Staleness	How out of date a feature or prediction is	16.1

Term	Meaning	Where
Standard deviation (std)	Typical distance of values from their mean; the square root of the variance	2.8
Standard error	Spread of a sample statistic, σ/√n for a mean	2.10
Standardisation	Rescaling to mean 0, std 1: z = (x − μ)/σ	2.8
State / action / reward	Situation the agent is in / choice it makes / number telling it how good the result was	14.1
Stateless service	Service that keeps no per-user memory between requests, so any copy can answer	16.6
Stateless vs stateful retraining	Training from scratch each time / continuing from the previous model	16.8
State-space model	Sequence architecture with a linear recurrent state that scales to long inputs (e.g. Mamba)	12.7
State-value / action-value function	Expected return from a state V(s) / from taking an action in a state Q(s, a)	14.3
Stationarity	Statistical properties (mean, variance) not changing over time	12.10
Statistical power	Chance a test detects an effect that really exists, 1 − β	2.11
Step-up authentication	Asking for an extra identity check (one-time code, 3-D Secure) when risk is uncertain	18.2
Sticky assignment	Keeping each user in the same experiment arm or model version across requests	16.6
Stochastic gradient descent (SGD)	Gradient descent using one example or a mini-batch per step	2.5
Stop words / stemming / lemmatisation	Dropping very common words / cutting words to a crude stem / mapping words to their dictionary form	12.8
Stratified split	Split keeping class proportions equal in every part	4.7
Streaming inference	Scoring events as they flow through a stream	16.5
Stride	How many pixels a kernel moves between positions	11.1
Subword tokenisation	Splitting rare words into frequent pieces so any text can be encoded with a fixed vocabulary	12.8
Successive halving / Hyperband	Tuning that gives many candidates a small budget and keeps only the best for the next, bigger round	6.9
Supervised learning	Learning from inputs paired with correct labels	1.4
Support (association rules)	Fraction of baskets containing an itemset	9.8
Support vector	Training point on or inside the margin that defines an SVM's boundary	8.2
Surrogate model	Cheap model that approximates an expensive one (e.g. score vs hyperparameters in Bayesian optimisation)	6.9
SVD (singular value decomposition)	Factorising any matrix as $U\Sigma V^T$ (rotate, stretch, rotate); keeping the top singular values gives the best low-rank approximation	2.3
SVM (support vector machine)	Classifier finding the maximum-margin hyperplane, optionally with kernels	8.2
SVR (support vector regression)	SVM for numbers: fits a tube of width ε and ignores errors inside it	8.2
Switchback test	Experiment alternating treatment and control over time slots, for marketplaces with interference	16.9
Tabular data	Data in rows and columns, like a spreadsheet or database table	7.1
Tanh	Activation squashing values into (−1, 1); zero-centred cousin of the sigmoid	10.2
Target encoding	Replacing a category with a smoothed, out-of-fold mean of the target	4.4
Target leakage	A feature that secretly contains the answer, e.g. a field filled in only after the outcome	1.10
Target network	Frozen copy of the Q-network used to compute stable TD targets in DQN	14.8
TD error (δ)	r + γV(s') − V(s): the surprise between predicted and observed value	14.6
Teacher forcing	Feeding the true previous token to the decoder during training	12.5
Temperature (softmax)	Divisor applied to scores before a softmax; low values make the distribution sharper	13.7
Temporal difference (TD) learning	Updating a value toward reward + γ·(estimate of next value) after every step	14.6
Tensor	n-dimensional array, the basic data type of deep-learning frameworks	11.1
TensorFlow / Keras	Google's deep-learning framework and its high-level model-building API	3.7
Test set	Data used once at the end to estimate real-world performance	1.6
Test statistic	Number computed from the data to judge a hypothesis (e.g. a t- or z-value)	2.11
Test-time augmentation (TTA)	Averaging predictions over several augmented copies of a test input	11.6
TF-IDF	Term frequency × inverse document frequency: down-weights words common across documents	4.6
Thompson sampling	Sampling each arm's payoff from its posterior and pulling the best sample	14.4
Threat model	Statement of what an attacker wants, knows and can do	19.7
Threshold (decision)	Score cut-off turning probabilities into decisions; sets the precision–recall trade-off	6.3
Threshold moving	Changing the score cut-off instead of the model to trade precision for recall	4.9
Throughput	Amount of work completed per unit of time (e.g. predictions per second)	16.1

Term	Meaning	Where
Tiling	Cutting a large image into overlapping pieces so a detector sees each at normal size	18.7
Time-based split	Training on the past and testing on the future	4.7
Timeout	Maximum wait before a call is abandoned and a fallback is used	16.2
Time series	Measurements ordered in time, such as daily sales	12.10
Time step	One position in a sequence	12.1
Token	Unit of text a model reads (a word, subword or character), mapped to an id	12.1
Tokenization	Splitting text into tokens mapped to integer ids from a vocabulary	12.8
Tombstone	Marker saying an index entry is deleted, used instead of immediate removal	18.6
TPE (tree-structured Parzen estimator)	Bayesian-optimisation method used by Optuna	6.9
TP / FP / FN / TN	Counts of true positives, false positives, false negatives and true negatives in a confusion matrix	6.3
TPR / FPR / FNR / PPV	True-positive rate (recall) / false-positive rate / false-negative rate / positive predictive value (precision)	6.3
Training-data extraction	Getting a generative model to output memorised training text or images	19.7
Training pipeline	Automated steps that turn raw data into a validated, registered model	16.1
Training–serving skew	Features computed differently in production than in training	16.2
Training set	The examples a model learns its parameters from	1.6
Transfer learning	Reusing a model pretrained on a large task as the starting point for yours	11.5
Transformer	Architecture of stacked self-attention and feed-forward blocks with residuals and LayerNorm	12.7
Transition probability	P(s' | s, a): chance of landing in s' after action a in state s	14.2
Translation equivariance / invariance	Shifting the input shifts the output the same way / leaves the output unchanged	11.1
Transpose	Flipping a matrix so rows become columns (A^T)	2.2
Trend / seasonality	Long-run rise or fall / repeating pattern over a fixed period	12.10
Triangle closing	Suggesting C to A when A–B and B–C are already connected	18.4
Truncated BPTT	Backpropagating through only the last k steps to save memory on long sequences	12.2
Trustworthiness	Score of how well a low-dimensional map keeps true neighbours together	9.6
t-SNE	Non-linear 2-D visualisation preserving local neighbourhoods	9.6
TTL (time to live)	How long a cached entry stays valid before it expires	17.4
Two-sided marketplace	Platform serving two groups (guests and hosts, users and organisers) whose interests both count	18.8
Two-stage vs one-stage detector	Propose regions then classify them (Faster R-CNN) / predict boxes in a single pass (YOLO)	11.7
Two-tower model	Separate user and item encoders whose embeddings meet at a dot product	13.7
Type I / Type II error	False positive (rejecting a true H_0) / false negative (missing a real effect)	2.11
UCB (upper confidence bound)	Bandit strategy adding an optimism bonus that shrinks as an arm is tried	14.4
UMAP	Non-linear dimensionality reduction for visualisation that can transform new points	9.6
Underfitting	Model too simple to capture the pattern; training and validation error both high	1.10
U-Net	Encoder–decoder segmentation network with concatenated skip connections	11.7
Universal approximation theorem	One hidden layer with enough units can approximate any continuous function on a bounded region	10.1
Unsupervised learning	Finding structure in data without labels	1.4
Upsampling	Increasing a feature map's spatial size, as in a segmentation decoder	11.7
User-based / item-based CF	Recommending from similar users' choices / from items similar to ones the user liked	13.4
Validation curve	Training and validation score vs one hyperparameter	6.8
Validation set	Data used to compare models and tune hyperparameters	1.6
Value formula	Product-owned weighted sum of predicted probabilities used as the final ranking score	17.2
Value function	See state-value / action-value function	14.3
Value iteration	Repeating the Bellman optimality update until values converge	14.5
Vanishing gradient	Gradients shrinking exponentially through many layers or time steps	10.6
Variance (model)	How much the model changes when trained on a different sample	6.7
Variance (statistical)	Average squared distance from the mean; how spread out values are	2.6
Vector	Ordered list of numbers, e.g. one example's features	2.1

Term	Meaning	Where
Vector database	Database that stores embeddings and answers nearest-neighbour queries	12.9
Vectorisation	Replacing Python loops with whole-array operations	3.2
Vector search (similarity search)	Finding the stored vectors closest to a query vector	8.1
Velocity feature	Count of recent activity in a short window (transactions on this card in 10 minutes)	18.2
Version skew	A new user tower querying an index built by an old item tower; silently degrades quality	13.7
View vs copy (arrays)	A view shares memory with the original array (edits show through); a copy is independent	3.2
VIF (variance inflation factor)	How much a coefficient's variance is inflated by correlation with other features; a multicollinearity check	5.10
Vision Transformer (ViT)	Transformer applied to image patches treated as tokens	11.4
Vocabulary	Fixed list of tokens a model knows, each with an id	12.8
Vocabulary mismatch	Query and document using different words for the same thing	17.3
Watermarking	Embedding a hidden, detectable mark in generated content or in a model	19.7
Weak learner	Model only slightly better than chance, e.g. a stump, used inside boosting	7.4
Weak supervision	Many noisy labelling functions combined by a label model into training labels	4.1
Weight decay	Shrinking weights each step, equivalent to an L2 penalty for SGD	10.7
Weights and bias (intercept)	Learned multipliers on each feature, plus a constant offset added to the sum	5.2
Weight sharing	Reusing the same weights at every position (convolutions, RNN steps)	11.1
White-box / black-box attack	Attacker knows the weights and code / can only send queries and read answers	19.7
Wide & Deep / DCN / DLRM	Ranking networks combining memorised feature crosses with deep layers (Google, Google, Meta)	13.8
Winsorizing (clipping)	Capping extreme values at chosen percentiles instead of deleting them	4.3
Word2vec	Shallow model learning word embeddings from context (skip-gram or CBOW)	12.9
Word embedding	Learned vector for a word, placing words used in similar contexts near each other	12.9
WordPiece / SentencePiece	Subword tokenisers used by BERT / by many multilingual models	12.8
Xavier initialisation	Weight variance 2/(fan_in + fan_out), designed for tanh and sigmoid	10.6
XGBoost	Gradient-boosting library with a regularised second-order objective	7.4
XOR problem	Output 1 when exactly one input is 1; not linearly separable, so one perceptron can't learn it	10.1
YOLO	One-stage detector predicting boxes and classes over a grid in a single pass	11.7
Zero-shot	Handling a task or class with no task-specific training examples	11.8
Zipf distribution	A few items are extremely common and most are rare (a long tail)	17.3
z-score	(x − mean)/std: how many standard deviations from the mean	2.8

Chapter TL;DR

- **20.1–20.4**: the workflow (frame → data → prepare → train → evaluate → deploy → monitor), the math you need (dot products, gradients, Bayes, entropy), the NumPy/pandas/scikit-learn API and data preparation — split first, pipeline everything, fit on training data only.
- **20.5–20.8**: every algorithm with its knobs and costs, every loss, every metric with a worked number, and how to validate and tune honestly (CV, nested CV, calibration, early stopping).
- **20.9–20.12**: deep learning (activations, optimizers, init, regularisation, the debugging checklist), CNN/RNN/attention formulas, recommenders (MF, two-tower, funnel, ANN) and the RL update rules.
- **20.13–20.15**: the 7-step system-design framework with napkin numbers and the thirteen case studies, then the production/MLOps and responsible-ML checklists to run before every launch.
- **20.16–20.17**: decision tables for picking an algorithm and a metric, and 50 interview questions with crisp answers.
- **20.18**: the glossary — follow each **§** back to the chapter where the idea is explained properly.